UNIVERSITÉ DE YAOUNDÉ I
****
ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ
****
DÉPARTEMENT DE GÉNIE INFORMATIQUE

UNIVERSITY OF YAOUNDE I
****
NATIONAL ADVANCED SCHOOL OF ENGINEERING OF YAOUNDE
****
DEPARTMENT OF COMPUTER ENGINEERING

Optimisation du traitement des queues de m-vecteurs dans les noyaux AVX2 de MAYO-C : l'approche A2

Mémoire de Master II

Présenté et soutenu par :

Marcel Eric Angoah

En vue de l'obtention du :

Master II en Génie Informatique

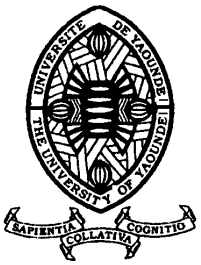
Sous la supervision de :

Dr Hervé Talé Kalachi

Année académique 2025-2026

Version provisoire (premier jet complet) — 25 octobre 2026





Optimisation du traitement des queues de m-vecteurs dans les noyaux AVX2 de MAYO-C : l'approche A2

Mémoire de Master II

Présenté et soutenu par :

Marcel Eric Angoah

En vue de l'obtention du :

Master II en Génie Informatique

Année académique 2025-2026

Soutenu le [date]

Dédicace

(*à compléter*)

Remerciements

« Une seule main ne peut nouer un fagot de bois. »
— Adage africain

Fort de cet adage, je prends la plume pour remercier celles et ceux dont le soutien m'a permis de mener à terme la rédaction de ce mémoire. Ce travail n'aurait pas été possible sans l'accompagnement de plusieurs personnes, à qui j'exprime ma sincère gratitude :

- **Dr Hervé Talé Kalachi**, mon encadrant, pour ses relectures exigeantes et les consignes méthodologiques qui ont structuré ce travail ;
- (*à compléter : membres du jury, enseignants du Département de Génie Informatique, collègues, famille, amis.*)

Abstract

MAYO is an Oil-and-Vinegar multivariate signature scheme submitted to post-quantum standardisation. In the official AVX2 C implementation, products of the public matrix P_1 by the vectors of V are computed with `vpshufb` lookups on packed $\mathbb{F}_{16}$ vectors. When the vector length m is not a multiple of 64, the official kernel covers the tail with an overlapping 32-byte load that duplicates work. We study a local change, called A2, which handles the 8-byte tail of each P_1 entry in a single 128-bit register and one xmm `vpshufb` on the low lane of the multiplication table, reducing the number of shuffles per scalar pair from 4 to 3 (MAYO-1) and from 6 to 5 (MAYO-5). We formalise the counting argument and its limits, establish the properties on which correctness rests, and measure the effect on a physical laptop with a paired protocol. Kernel gains are 12–29 %; the full-signature gain is 10.3 % for MAYO-1 but only 1.3 % for MAYO-5. Static analysis shows that in the signature context GCC 16.2.0 emits extra 32-byte stores in several loops, asymmetrically between the two versions, consistently with the measured loss; causality is not established. GCC's dumps show that the extra stores are per-iteration stores of the accumulator arrays that are not moved out of the loop; SRA is ruled out, and why the compiler makes this choice remains open.

Résumé

MAYO est un schéma de signature multivariée de type Oil-and-Vinegar, candidat à la standardisation post-quantique. Dans l'implémentation C officielle (MAYO-C) pour AVX2, les produits de la matrice publique P_1 par les vecteurs de V sont calculés par des instructions `vpshufb` sur des m-vecteurs de $\mathbb{F}_{16}$ empaquetés à deux éléments par octet. Lorsque m n'est pas un multiple de 64, le noyau officiel couvre la fin du vecteur par un chargement de 32 octets qui *recouvre* le précédent, ce qui duplique du calcul : 48 éléments sur 128 pour MAYO-1 ($m = 80$) et 50 sur 192 pour MAYO-5 ($m = 142$).

Ce mémoire étudie une modification locale, nommée A2, qui traite la queue de 8 octets de chaque entrée de P_1 dans un seul registre 128 bits : un seul `vpshufb` xmm sur la voie basse de la table de multiplication remplace deux `vpshufb` ymm. Le nombre de `vpshufb` par paire de scalaires passe de 4 à 3 (MAYO-1) et de 6 à 5 (MAYO-5). Nous formalisons le comptage (borne, réalisation, limites), établissons les propriétés dont dépend la correction, et mesurons l'effet sur un portable physique (Intel Core i9-9880H, GCC 16.2.0) avec un protocole apparié et l'unité statistique « processus ».

Les résultats établis sont les suivants. Sur les noyaux isolés, A2 réduit le temps de 19–22 % (`P1_times_Vt`) et de 12–29 % (`P1t_times_Vt`). Sur la signature complète, le gain est de 10,3 % pour MAYO-1 et de seulement 1,3 % pour MAYO-5, bien qu'il atteigne 7,3 % pour MAYO-5 lorsque toutes les sondes de mesure sont actives. L'analyse statique du code généré montre que, dans le contexte de la signature sans toutes les sondes, GCC 16.2.0 émet une écriture mémoire de 32 octets supplémentaire par accumulateur dans plusieurs boucles, de façon asymétrique entre les deux versions ; les temps mesurés varient de manière cohérente avec cette asymétrie, mais la causalité n'est pas démontrée et un modèle à un paramètre surestime le gain d'environ 40 %. L'analyse statique des sorties de GCC montre que ces écritures sont les stockages des tableaux d'accumulateurs maintenus à chaque itération au lieu d'être sortis de la boucle, et écarte l'hypothèse d'un rôle de la passe SRA ; la raison de cette décision du compilateur reste une question ouverte.

Mots-clés : MAYO, signatures multivariées, cryptographie post-quantique, AVX2, `vpshufb`, $\mathbb{F}_{16}$, optimisation, évaluation de performance.

Liste des abréviations

AVX2 : *Advanced Vector Extensions 2* (instructions SIMD 256 bits)

CT : *Constant Time* (temps constant)

DSE : *Dead Store Elimination* (passe de GCC)

FRE : *Full Redundancy Elimination* (passe de GCC)

GCC : *GNU Compiler Collection*

GFNI : *Galois Field New Instructions*

IC95 : intervalle de confiance à 95 %

KAT : *Known-Answer Test*

LIM : *Loop Invariant Motion* (passe de GCC)

MAYO : schéma de signature multivariée de type Oil-and-Vinegar

NIST : *National Institute of Standards and Technology*

OV, UOV : (*Unbalanced*) *Oil and Vinegar*

PRE : *Partial Redundancy Elimination* (passe de GCC)

SIMD : *Single Instruction, Multiple Data*

SRA : *Scalar Replacement of Aggregates* (passe de GCC)

TSC : *Time Stamp Counter*

Glossaire

- **m-vec :** vecteur de m éléments de $\mathbb{F}_{16}$, empaqueté à deux éléments par octet.
- **Limbe :** mot de 64 bits d'un accumulateur.
- **Voie (*lane*) :** moitié de 128 bits d'un registre ymm.
- **Table de multiplication (multab) :** table de 16 octets donnant, pour une paire de scalaires, les deux produits par un quartet.
- **Queue :** fin d'un m-vec qui ne remplit pas un registre complet de 32 octets.
- **Recouvrement :** chargement reculé du dernier registre, qui retraite des octets déjà traités.
- **SAFE :** variante d'A2 qui ne lit rien au-delà de l'entrée de P_1.
- **Tick TSC :** unité du compteur d'horodatage (`rdtscp`) ; ce n'est pas un cycle cœur.
- **Processus :** unité statistique : une exécution du banc, 3 001 paires.
- **Doublon :** écritures de 32 octets supplémentaires par accumulateur dans le code généré, selon la construction.
- **E1, E1b :** analyses statiques du code généré (E1b : origine du doublon, exécutée le 25 octobre 2026 sous GCC 16.2.0).
- **Fait, inférence, hypothèse :** statuts des énoncés, notés **[F]**, **[I]**, **[H]**.

Table des matières

Dédicace i

Remerciements ii

Abstract iii

Résumé iv

Liste des abréviations v

Glossaire vi

Liste des figures xi

Liste des tableaux xii

Introduction générale 1

Contexte et motivation . . . 1
Problématique . . . 1
Objectifs de l'étude . . . 2
Méthodologie de recherche . . . 2
Intérêt de l'étude . . . 3
Structure du mémoire . . . 3

1 Fondements et revue de littérature 5

1.1 Fondements théoriques . . . 5
1.1.1 Le schéma MAYO . . . 5
1.1.1.1 Des signatures multivariées à Oil-and-Vinegar . . . 5
1.1.1.2 Le schéma MAYO . . . 5
1.1.1.3 Évaluation par produits de matrices . . . 6
1.1.1.4 Algorithmes (version simplifiée) . . . 6

1.1.2 La signature dans le code de référence . . . 7
1.1.2.1 Le déroulement d'une tentative . . . 7
1.1.2.2 Où sont les noyaux étudiés . . . 7
1.1.2.3 Rôle de P_1 et paramètres . . . 8
1.1.3 Représentation des données . . . 8
1.1.3.1 Le corps $\mathbb{F}_{16}$ et les m-vecteurs . . . 8
1.1.3.2 Empaquetage en quartets . . . 8
1.1.4 Arithmétique $\mathbb{F}_{16}$ par `vpshufb` . . . 8
1.1.4.1 Rappel : le registre AVX2 et `vpshufb` . . . 9
1.1.4.2 Principe . . . 9
1.1.4.3 Deux produits par recherche . . . 9
1.1.4.4 Recouvrement en fin de vecteur . . . 9
1.1.5 Panorama des analyses menées . . . 9
1.2 Revue de littérature . . . 10
1.2.1 Implémentations existantes . . . 10
1.2.2 Canaux auxiliaires et temps constant . . . 11
1.3 Évaluation et positionnement . . . 11

2 Méthodologie **13**
2.1 Conception d'A2 : comptage, réalisation, hypothèses et limites . . . 13
2.1.1 Modèle de comptage . . . 13
2.1.2 Énoncés . . . 14
2.1.3 Conception d'A2 . . . 15
2.1.3.1 Structure du noyau . . . 15
2.1.3.2 Variante par défaut et variante SAFE (MAYO-5) . . . 15
2.1.3.3 Préconditions . . . 15
2.1.4 La borne n'implique pas l'accélération . . . 16
2.2 Propriétés de correction et de sûreté . . . 16
2.2.1 Bornes de lecture de P_1 . . . 16
2.2.2 Bornes d'écriture . . . 17
2.2.3 Invariant des deux voies des tables . . . 17
2.2.4 Influence des quartets de bourrage . . . 18
2.2.5 Temps constant : ce qui est établi et ce qui ne l'est pas . . . 18
2.2.6 Correction fonctionnelle . . . 19
2.3 Protocole expérimental . . . 19
2.3.1 Plateforme . . . 19

2.3.2 Code comparé . . . 19
2.3.3 Protocole de mesure apparié . . . 20
2.3.4 Unité statistique et intervalles . . . 20
2.3.5 Sessions . . . 20
2.3.6 Instrumentation de décomposition . . . 21
2.3.7 Analyse statique du code généré (E1) . . . 21
2.3.8 Protocole E1b (analyse statique de l'origine des écritures) . . . 21
2.3.9 Traçabilité et reproduction . . . 22
2.3.10 Menaces à la validité identifiées a priori . . . 22

3 Résultats et discussion **23**
3.1 Correction . . . 23
3.2 Gains sur les noyaux et sur la signature (S1, S2) . . . 23
3.3 Effet d'ordre . . . 24
3.4 Anomalie de MAYO-5 . . . 25
3.4.1 Toutes sondes actives (PH1) . . . 25
3.4.2 Le gain avec toutes les sondes n'est pas le gain sans sonde (CT1) 25
3.4.3 Mesure directe, sans soustraction de médianes (config. `mcalc`) . 25
3.5 Analyse statique (E1) . . . 26
3.5.1 Comptage de base . . . 26
3.5.2 Écritures supplémentaires en contexte de signature . . . 27
3.6 Instructions et temps . . . 27
3.7 Analyse statique E1b . . . 28
3.7.1 Contrôles . . . 28
3.7.2 Désactivation de passes, une à la fois . . . 28
3.7.3 Ce que montrent les sorties de GCC . . . 29
3.7.4 Interprétation autorisée . . . 29
3.8 Comparaison avec la VM . . . 30
3.9 Discussion critique . . . 30
3.9.1 Bilan épistémique . . . 30
3.9.2 Pourquoi le gain se dilue . . . 30
3.9.3 Ce que la concordance statique/dynamique prouve et ne prouve pas . . . 30
3.9.4 Pourquoi MAYO-1 et MAYO-5 se comportent différemment . . . 31
3.9.5 Pistes explorées puis écartées . . . 31
3.9.6 Limites de la validation . . . 32
3.9.7 Sur l'intégration dans MAYO-C . . . 32

Conclusion générale et perspectives **34**

Références **36**

A Inventaire des archives et reproduction **37**

A.1 Versions définitives à citer 37

A.2 Identification exacte du code 38

A.3 Reproduire les analyses 38

A.4 Refaire les mesures 38

B Identification des noyaux dans le code généré (GCC 16.2.0, MAYO-5) **39**

Table des figures

1.1 Une tentative de signature dans le code de référence. Les noyaux optimisés se trouvent dans l'étape en rouge, répétée à chaque tentative. 7

1.2 Chargements pour une entrée de P_1 de MAYO-5 (octets). Le noyau officiel recouvre l'intervalle $[40, 64)$ et dépasse l'entrée d'un octet ; A2 SAFE lit exactement $[0, 71)$. 10

3.1 Rapports A2/officiel mesurés (S1). Le gain se dilue de la couche noyau à la signature complète, fortement pour MAYO-5. 24

3.2 MAYO-5, rapport `mayo_sign` A2/officiel selon la configuration de sondes (CT1). Seule la configuration « toutes les sondes » change le résultat. 26

Liste des tableaux

1 Statut des contributions. 4

1.1 Paramètres des noyaux étudiés. m : longueur du m-vecteur ; $b = m/2$: octets par m-vecteur ; L : limbes de 64 bits d'un accumulateur ; K : nombre de vecteurs de V ; V : dimension de la partie vinaigre ; $N_{\text{ent}} = V(V+1)/2$: nombre d'entrées de P_1. 8

1.2 Situation d'A2 par rapport aux sources consultées. 12

2.1 Application aux paramètres actuels. 14

2.2 Rapport attendu par le seul débit de `vpshufb` et rapport mesuré (S1, A2/officiel, noyaux isolés). 16

2.3 Sessions de mesure sur machine physique (dates UTC). 20

2.4 Issues possibles d'E1b et conclusions autorisées (fixées avant exécution). 22

3.1 Rapport A2/officiel, médiane des processus [IC95 bootstrap entre processus]. S1 : 20 processus ; S2 : 5 processus. Gain = 1 − rapport. . . 23

3.2 Rapport A2/officiel selon l'ordre d'exécution, et écart par processus [IC95]. 25

3.3 Cinq configurations du même programme (10 processus chacune, graines 41–50, un démarrage). Ticks par signature. 25

3.4 Décomposition directe de la signature MAYO-5 (ticks par signature). . 26

3.5 Instructions de la boucle interne par entrée de P_1 (MAYO-5) selon la construction. 27

3.6 E1b : boucles de travail selon l'option ajoutée (instructions / écritures de 32 ou 16 octets). « Dist. » : distance entre histogrammes de mnémoniques hors boucles de travail, par rapport à `base`. 28

3.7 Établi, inféré, hypothétique. 33

B.1 Instructions / écritures de 32 octets des boucles de travail selon la construction. 39

Contexte et motivation

L'algorithme de Shor [1] rend les signatures fondées sur la factorisation et le logarithme discret vulnérables à un ordinateur quantique de grande taille. Le NIST a publié en 2024 les premières normes post-quantiques (FIPS 203, 204 et 205 [8]) et poursuit un appel à signatures additionnelles [9], dans lequel figurent des schémas multivariés. Parmi eux, MAYO [6] construit une signature de type Oil-and-Vinegar (OV) [2, 3, 4] à clé publique compacte : l'application publique est une application quadratique d'un petit espace d'huile, « fouettée » (*whipped*) pour produire une application plus grande. L'intérêt pratique du schéma est la petite taille des clés et des signatures ; son coût de calcul est dominé par de l'arithmétique sur $\mathbb{F}_{16}$.

Les schémas multivariés ont connu des cryptanalyses marquantes [5], ce qui impose une lecture prudente de toute affirmation de sécurité ; ce mémoire ne traite pas de sécurité algébrique. Il traite de *performance d'implémentation* et de *propriétés d'implémentation* (correction, accès mémoire, temps constant) d'une optimisation locale du code C officiel.

Parcours du travail et recentrage. Le travail a débuté par une implémentation autonome (arithmétique $\mathbb{F}_{16}$ à tables, étude d'une fuite par canal auxiliaire de cache sur les tables, élimination de Gauss–Jordan à temps constant), documentée dans des chapitres antérieurs et motivée par les attaques temporelles de cache [10, 11]. À la demande de l'encadrant, le travail a ensuite été recentré sur l'implémentation C *officielle* (MAYO-C) pour mesurer des gains qui soient comparables à l'existant. Ce mémoire porte exclusivement sur cette seconde phase. L'étude préliminaire n'est pas reprise, sauf quand un résultat négatif de cette phase éclaire une décision (section 3.9).

Problématique

Dans l'implémentation AVX2 officielle, un m-vecteur de $\mathbb{F}_{16}$ est stocké à deux éléments par octet ($m/2$ octets) et multiplié par des scalaires à l'aide de `vpshufb` : une instruction effectue 32 recherches dans une table de 16 octets et fournit, pour chaque élément, les produits par *deux* scalaires à la fois. Un registre de 256 bits couvre donc 64 éléments. Quand m n'est pas un multiple de 64, le code officiel couvre la fin du vecteur par un chargement de 32 octets reculé qui *recouvre* le précédent. Ce recouvrement est

documenté dans l'article d'optimisation [7] pour $m = 96$ (MAYO-3) : il « duplique » des opérations.

Pour les paramètres considérés ici, la duplication est importante : 48 éléments recalculés sur 128 pour MAYO-1 ($m = 80$, soit 37,5 %) et 50 sur 192 pour MAYO-5 ($m = 142$, soit 26 %). La question centrale est :

Question centrale. La duplication de calcul due au recouvrement en fin de m-vecteur peut-elle être réduite par un traitement dédié de la queue, quelle en est la limite exacte en nombre de `vpshufb`, et ce gain se traduit-il, de façon robuste et démontrable, en gain sur le temps de signature complet ?

Cette formulation sépare trois choses souvent confondues : (i) une *borne de comptage* (mathématique, dans un modèle précis), (ii) sa *réalisation* par un algorithme (à montrer et à tester) et (iii) l'*effet sur le temps* (expérimental, dépendant du compilateur et de la machine).

Objectifs de l'étude

QR1 Quel est le nombre minimal de `vpshufb` par paire de scalaires pour un m-vecteur, dans la classe des traitements à tables de 16 octets, et pour quels m le code officiel est-il déjà optimal ?

QR2 Existe-t-il une réalisation sûre (sans lecture hors de l'entrée, sans dépendance au secret dans les adresses) de cette borne pour MAYO-1 et MAYO-5 ?

QR3 Quel gain mesure-t-on sur les noyaux `P1_times_Vt` et `P1t_times_Vt`, puis sur `mayo_sign`, sur une machine physique, avec un protocole dont les limites sont quantifiées ?

QR4 Si le gain en contexte diffère du gain isolé, que montre l'analyse du code généré, et qu'est-ce qui reste à démontrer ?

Les objectifs en découlent : **O1** formaliser le comptage (borne, réalisation, limite) ; **O2** documenter la conception d'A2 et les propriétés dont dépend sa correction ; **O3** mesurer avec un protocole reproductible ; **O4** caractériser l'écart entre gain isolé et gain en contexte ; **O5** situer le travail par rapport aux implémentations existantes sans revendiquer plus que ce que les sources consultées permettent.

Conformément à la règle de rédaction adoptée, le tableau 1 sépare ce qui est acquis de ce qui reste ouvert.

Méthodologie de recherche

La démarche est expérimentale et déductive. Le comptage est établi par preuve dans un modèle défini ; la réalisation est justifiée par le code et testée ; les gains sont

mesurés sur machine physique avec un protocole apparié, dont l'unité statistique est le processus ; le code généré par le compilateur est analysé statiquement pour interpréter les écarts. Chaque résultat est étiqueté selon son statut (fait, inférence, hypothèse), et les scripts, données brutes et empreintes sont déposés pour permettre la reproduction.

Le travail est limité aux noyaux `P1_times_Vt` et `P1t_times_Vt` pour MAYO-1 et MAYO-5. MAYO-2 ($m = 64$) et MAYO-3 ($m = 108$) servent de témoins : pour eux, aucun gain n'est possible dans la classe étudiée (proposition 2.3). Aucune autre optimisation n'est proposée, les noyaux optimisés ne sont pas modifiés pendant l'analyse, et la décision d'intégration dans MAYO-C est volontairement différée. L'absence d'une technique dans les sources consultées ne prouve pas qu'elle n'existe pas ailleurs : aucune nouveauté absolue n'est revendiquée.

Les énoncés portant sur des résultats sont étiquetés : **[F]** *fait* (lu dans le code généré ou dans des mesures déposées), **[I]** *inférence* tirée des faits, **[H]** *hypothèse* non testée. Les propriétés de sûreté du section 2.2 utilisent **[C]** (justifiée par le code) et **[E]** (vérifiée expérimentalement) ; une propriété **[E]** seule n'est pas une démonstration. Les durées sont en ticks du compteur TSC (rdtscp), non en cycles cœur.

Intérêt de l'étude

Pour les mainteneurs et les utilisateurs de MAYO-C, l'étude fournit une borne précise sur ce qu'une optimisation du recouvrement peut gagner en nombre de `vpshufb`, une réalisation vérifiée pour deux jeux de paramètres, et surtout une mise en garde méthodologique : un gain mesuré sur un noyau isolé ne se transfère pas automatiquement à la signature, car le code généré par le compilateur dépend du contexte. Pour la méthode, le mémoire propose un protocole de mesure et d'analyse statique qui sépare les faits des hypothèses et qui est reproductible.

Structure du mémoire

Le mémoire comprend trois chapitres, encadrés par cette introduction et par une conclusion. Le chapitre 1 présente MAYO, l'arithmétique par `vpshufb` et les implémentations existantes, et situe A2. Le chapitre 2 formalise le comptage et décrit A2 (section 2.1), établit ses propriétés de correction et de sûreté (section 2.2) et détaille le protocole expérimental (section 2.3). Le chapitre 3 présente les résultats et les discute en séparant faits, inférences et hypothèses. La conclusion récapitule contributions, limites et perspectives.

Tab. 1 : Statut des contributions.

	Contribution	Statut
C1	Comptage : borne $\lceil m/32 \rceil$, schéma officiel $2\lceil m/64 \rceil$, écart $\Delta(m)$ (prop. 2.3–2.5)	établi (preuve)
C2	Réalisation A2 (queue $\leq$ 16 éléments) pour MAYO-1 et MAYO-5, variante SAFE sans lecture hors de P1	établi (code, KAT, tests)
C3	Gains sur noyaux isolés (12–29 %) et sur `mayo_sign` (MAYO-1 : 10,3 % ; MAYO-5 : 1,3 %) sur une machine, un compilateur	établi, avec limites
C4	Localisation de la perte de gain de MAYO-5 dans `m_calculate` en contexte de signature	établi (mesure directe)
C5	Écritures supplémentaires asymétriques dans le code généré par GCC 16.2.0, cohérentes avec C4 ; identifiées (E1b) à des stockages de `temp[]/tt[]` dans la boucle ; SRA écartée	fait statique établi ; causalité sur le temps *non* établie
C6	Dépôt public de scripts, données brutes et empreintes, reproduisant les analyses (9/9)	établi
Q1	Pourquoi GCC ne sort pas ces stockages de la boucle sans toutes les sondes (passe responsable)	*ouvert* (dumps `lim/pre` non produits)
Q2	Lien causal écritures $\rightarrow$ perte de gain, avec un modèle qui n'en surestime pas l'effet	*ouvert*
Q3	Variante pour queues de 17 à 32 éléments ; autres compilateurs, autres microarchitectures	*ouvert* (non implémenté)
Q4	Temps constant au sens fort (preuve formelle)	*non fourni*

Chapitre 1

Fondements théoriques et revue de littérature

Ce chapitre donne au lecteur ce qui est nécessaire pour suivre la suite sans autre source : le schéma de signature MAYO et ses algorithmes (section 1.1.1), leur traduction dans le code de référence et la place des noyaux étudiés (section 1.1.2), la représentation des données et l'arithmétique par `vpshufb` sur $\mathbb{F}_{16}$ (section 1.1.4), un panorama des analyses menées (section 1.1.5), puis les implémentations existantes et le positionnement de l'optimisation étudiée (sections 1.2 et suivantes).

1.1 Fondements théoriques

1.1.1 Le schéma MAYO

1.1.1.1 Des signatures multivariées à Oil-and-Vinegar

Une signature multivariée repose sur une application publique quadratique $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$, formée de m polynômes homogènes de degré 2 en n variables. Une signature d'un message M est un point s tel que $\mathcal{P}(s) = H(M)$, où H est une fonction de hachage salée à valeurs dans $\mathbb{F}_q^m$; la vérification consiste donc à évaluer $\mathcal{P}$. Le schéma Oil-and-Vinegar (OV) [2, 3, 4] cache une structure : la clé secrète est un sous-espace $\mathcal{O} \subset \mathbb{F}_q^n$ («l'huile») de dimension o sur lequel $\mathcal{P}$ s'annule. Dans OV, $o = m$: pour signer, on tire au hasard les composantes «vinaigre» $v \in \mathbb{F}_q^{n-o}$, et il reste à résoudre un *système linéaire* de m équations en les o composantes d'huile. L'inconvénient d'OV est la taille de ses clés.

1.1.1.2 Le schéma MAYO

MAYO [6, 7] choisit un espace d'huile *trop petit* ($o < m$), ce qui réduit fortement les clés mais interdit la résolution directe. La parade est le «fouettage» : l'application publique

$\mathcal{P}$ est étendue en une application $\mathcal{P}^\star : \mathbb{F}_q^{kn} \to \mathbb{F}_q^m$ à k blocs de n variables,

$$\mathcal{P}^\star(x_1, \ldots, x_k) = \sum_{i=1}^{k} E_{ii}\, \mathcal{P}(x_i) + \sum_{i=1}^{k} \sum_{j=i+1}^{k} E_{ij}\, \mathcal{P}'(x_i, x_j), \qquad \mathcal{P}'(x, y) = \mathcal{P}(x+y) - \mathcal{P}(x) - \mathcal{P}(y),$$

où les matrices publiques $E_{ij} \in \mathbb{F}_q^{m \times m}$ représentent la multiplication par des puissances de X modulo un polynôme irréductible de degré m. Une signature est $S = (s_1, \ldots, s_k)$, valide si $\mathcal{P}^\star(S) = H(M)$. Ici $q = 16$. Les paramètres m, k, o, n définissent quatre jeux (MAYO-1, 2, 3 et 5) selon le niveau de sécurité NIST [9].

1.1.1.3 Évaluation par produits de matrices

Les m polynômes de $\mathcal{P}$ s'écrivent $p_\ell(x) = x^\top P_\ell\, x$, avec P_ℓ triangulaire supérieure, découpée en blocs

$$P_\ell = \begin{pmatrix} P_\ell^{(1)} & P_\ell^{(2)} \\ 0 & P_\ell^{(3)} \end{pmatrix},$$

où $P_\ell^{(1)}$ est de taille $v \times v$ avec $v = n - o$ (noté V dans le code et au tableau 1.1), $P_\ell^{(2)}$ de taille $v \times o$ et $P_\ell^{(3)}$ de taille $o \times o$. Les blocs $P^{(1)}$ et $P^{(2)}$ sont dérivés d'une graine par une fonction d'expansion à base d'AES ; seul $P^{(3)}$ est stocké dans la clé publique [7]. Si S est la matrice dont les colonnes sont $s_1, \ldots, s_k$, tout se déduit de $S^\top P_\ell S$: $\mathcal{P}(s_i)_\ell$ est son élément diagonal (i, i) et $\mathcal{P}'(s_i, s_j)_\ell$ la somme des éléments (i, j) et (j, i). *Les produits matrice × matrice dominent donc le coût*, ce qui motive leur optimisation [7].

1.1.1.4 Algorithmes (version simplifiée)

KeyGen. Tirer `seed_sk` ; en dériver `seed_pk` et $O \in \mathbb{F}_{16}^{v \times o}$; développer $\{P_\ell^{(1)}, P_\ell^{(2)}\}$ depuis `seed_pk` ; calculer $P_\ell^{(3)} = \mathrm{Upper}(-O^\top (P_\ell^{(1)} O - P_\ell^{(2)}))$. Clé publique : `seed_pk` et $\{P_\ell^{(3)}\}$; clé secrète : `seed_sk`.
Sign(M). (1) Tirer un sel et calculer $t = H(M \,\|\, \mathrm{sel}) \in \mathbb{F}_{16}^m$. (2) Tirer $V \in \mathbb{F}_{16}^{k \times v}$ (les « vinaigres »). (3) Pour chaque $\ell < m$: $L_\ell = (P_\ell^{(1)} + P_\ell^{(1)\top})\, O + P_\ell^{(2)}$; $M_\ell = V\, L_\ell$; $Y_\ell = V\, P_\ell^{(1)}\, V^\top$. (4) Assembler le système linéaire $Ax = y$, où A provient des M_ℓ et $y = t +$ (combinaison des Y_ℓ). (5) Si le système n'a pas de solution, retourner à l'étape 2 (nouveaux vinaigres) ; sinon tirer une solution x au hasard. (6) Signature : $S = (V + (OX)^\top, X)$, où X est x mis en matrice $k \times o$.
Verify. Redévelopper $P^{(1)}, P^{(2)}$; calculer $t = H(M \,\|\, \mathrm{sel})$; calculer $Y_\ell = S \begin{pmatrix} P_\ell^{(1)} & P_\ell^{(2)} \\ 0 & P_\ell^{(3)} \end{pmatrix} S^\top$, combiner en $\mathcal{P}^\star(S)$; accepter si le résultat égale t.

La signature fait donc intervenir : des produits de matrices (M_ℓ et Y_ℓ, à partir de la clé publique étendue $P^{(1)}$, $P^{(2)}$), la résolution d'un système linéaire par élimination de Gauss en temps constant, et des boucles de reprise. Selon [7], les évaluations partielles de $\mathcal{P}$ et l'élimination de Gauss sont les étapes les plus coûteuses ; l'assemblage de A et de y ne l'est que marginalement.

1.1.2 La signature dans le code de référence

1.1.2.1 Le déroulement d'une tentative

Dans l'arbre de référence (annexe A), `mayo_sign_signature` calcule le sel et t, puis exécute une boucle d'au plus 256 tentatives (octet de compteur). Chaque tentative enchaîne les étapes suivantes (figure 1.1) :

1. tirer et décoder les vinaigres V (SHAKE256) ;
2. `m_calculate_M_and_VPV` : calculer les matrices M_ℓ et tous les produits $v_i P^{(1)} v_j$;
3. `compute_rhs` (second membre y) puis `compute_A` (matrice A) ;
4. `sample_solution` : résoudre $Ax = y$; en cas d'échec, nouvelle tentative ;
5. `m_compute_s` : assembler la signature $s_i = [\, v_i + Ox_i \mid x_i \,]$.

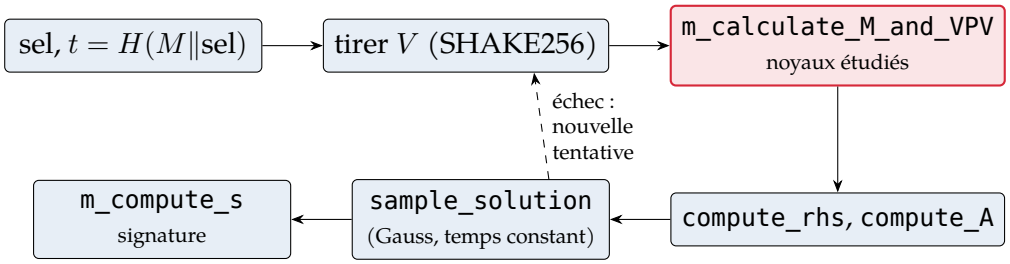


Fig. 1.1 : Une tentative de signature dans le code de référence. Les noyaux optimisés se trouvent dans l'étape en rouge, répétée à chaque tentative.

1.1.2.2 Où sont les noyaux étudiés

Dans `m_calculate_M_and_VPV` (chemin à L implicite, celui où les deux noyaux interviennent), l'ordre des opérations est le suivant : construction des tables de multiplication de V (`mayo_V_multabs`), puis :

- `P1_times_Vt` : accumule $P^{(1)} \cdot V^\top$ dans un tampon ; `Vt_times_Pv` en déduit $VP^{(1)}V^\top$, c'est-à-dire tous les $v_i P^{(1)} v_j$;
- `P1t_times_Vt` : accumule la contribution transposée $P^{(1)\top}V^\top$ sur le même tampon, qui devient $(P^{(1)}+P^{(1)\top})V^\top$ (la diagonale s'annule en caractéristique 2) ; le calcul de M_ℓ réutilise ce produit, puis `Ut_times_O` et `Vt_times_L` complètent M_ℓ.

Les noyaux `P1_times_Vt` et `P1t_times_Vt` sont donc des produits de la *partie vinaigre de la clé publique* $P^{(1)}$ par les multiples de V. Ce sont eux, et eux seuls, que ce mémoire optimise (variante A2) pour MAYO-1 et MAYO-5.

1.1.2.3 Rôle de P_1 et paramètres

P_1 est une matrice triangulaire supérieure de m-vecteurs de $\mathbb{F}_{16}^m$; elle est stockée à plat, une entrée après l'autre, à la position $e \cdot b$ pour l'entrée numéro e (disposition empaquetée du flux de clé). Chaque noyau accumule, pour chaque entrée de P_1 et chaque vecteur de V, le produit d'un scalaire de $\mathbb{F}_{16}$ par le m-vecteur de l'entrée, dans un accumulateur de L limbes. Les bornes de boucle (V, K) et les adresses de chargement ($e \cdot b$) ne dépendent que de paramètres publics. Les grandeurs utiles sont au tableau 1.1.

TAB. 1.1 : Paramètres des noyaux étudiés. m : longueur du m-vecteur ; $b = m/2$: octets par m-vecteur ; L : limbes de 64 bits d'un accumulateur ; K : nombre de vecteurs de V ; V : dimension de la partie vinaigre ; $N_{\text{ent}} = V(V+1)/2$: nombre d'entrées de P_1.

Paramètre	m	b	L	K	V	N_{ent}	$m \bmod 64$
MAYO-1	80	40	5	10	80	3 240	16
MAYO-2	64	32	4	–	73	–	0
MAYO-3	108	54	–	–	–	–	44
MAYO-5	142	71	9	12	142	10 153	14

1.1.3 Représentation des données

1.1.3.1 Le corps $\mathbb{F}_{16}$ et les m-vecteurs

$\mathbb{F}_{16}$ est réalisé comme $\mathbb{F}_2[x]/(x^4+x+1)$ (commentaire du code, `simple_arithmetic.h`) : un élément tient dans un *quartet* (4 bits) et l'addition est le OU exclusif. L'unité de calcul de MAYO est le *m-vecteur* : le vecteur $(a_0[i,j], \ldots, a_{m-1}[i,j])$ des éléments de même position dans les m matrices P_ℓ. On calcule donc en parallèle sur les m composantes, ce qui se prête naturellement à la vectorisation [7].

1.1.3.2 Empaquetage en quartets

Un m-vecteur est stocké sur $b = m/2$ octets : deux éléments par octet, l'élément d'indice pair dans le quartet de poids faible. Les accumulateurs sont découpés en *limbes* de 64 bits, soit 16 éléments par limbe ; il faut $L = \lceil m/16 \rceil$ limbes, avec éventuellement des quartets de bourrage dans le dernier limbe ($m = 142 : L = 9$, 144 positions, deux de bourrage). Cette représentation par quartets a été proposée dans [7] à la place de la représentation « bitsliced » (par plans de bits) de la première spécification ; elle permet d'utiliser des instructions de permutation d'octets pour multiplier, au prix d'un changement de format de clé.

1.1.4 Arithmétique $\mathbb{F}_{16}$ par `vpshufb`

1.1.4.1 Rappel : le registre AVX2 et `vpshufb`

Un registre AVX2 `ymm` contient 256 bits, soit deux *voies* de 128 bits (`xmm` désigne la moitié basse) ; la plupart des instructions, dont `vpshufb`, opèrent indépendamment sur chaque voie. `vpshufb` reçoit une table de 16 octets (par voie) et un registre d'index : pour chaque octet d'index, il écrit l'élément de la table désigné par les 4 bits bas de l'index, ou 0 si le bit 7 de l'index est positif. Un `ymm` fait donc 32 recherches en une instruction.

1.1.4.2 Principe

Avec des éléments de 4 bits, la multiplication d'un vecteur par un scalaire c est une recherche dans une table de 16 entrées $x \mapsto c \cdot x$. L'instruction `vpshufb` (`_mm256_shuffle_epi8`) réalise, pour chaque octet d'un registre d'index, une recherche dans une table de 16 octets (celle de la voie de 128 bits qui contient l'octet) ; si le bit 7 de l'index est nul, l'indice est le quartet bas, sinon la sortie est 0. C'est la méthode des « quatre Russes » [12], qui échange des multiplications contre des lectures de tables : pour $A \cdot B$, on découpe A en bandes de t colonnes et B en bandes de t lignes, on précalcule pour chaque bande de B toutes les combinaisons linéaires de ses lignes (16^t entrées), puis chaque ligne de A sert d'indice. Ici la table tient dans un registre, donc $t = 1$ (voir aussi [13] pour $\mathbb{F}_2$) ; les tables dépendent de V (secret) mais ne servent que d'opérande de données, jamais d'adresse.

1.1.4.3 Deux produits par recherche

Pour une paire de scalaires (c_{2p}, c_{2p+1}), la table $T_p[x] = (c_{2p}x) \oplus \big((c_{2p+1}x) \ll 4\big)$ fournit *les deux* produits dans un seul octet. Le code stocke T_p dans un registre 256 bits dont les deux voies sont égales (lemme 2.7). Un registre de données de 32 octets contient 64 éléments ; il faut séparer les quartets impairs et pairs (ET avec `0x0f`, décalage de 4) et donc deux `vpshufb` par registre et par paire. Les accumulateurs impairs et pairs sont maintenus séparés et ne sont « désintriqués » qu'à la fin de la ligne, technique décrite dans l'article *Nibbling MAYO* [7].

1.1.4.4 Recouvrement en fin de vecteur

Pour $m \le 64$, $64 < m \le 128$ et $128 < m \le 192$, le code officiel utilise respectivement 1, 2 et 3 chargements de 32 octets, le dernier étant *reculé* pour finir à l'octet b (adresse `vec + M_VEC_LIMBS_MAX - 4` limbes) ; les octets en commun sont donc traités deux fois et les stockages finaux se recouvrent avec des valeurs identiques. Le décompte pour MAYO-5 est le suivant : chargements $[0, 32)$, $[32, 64)$ et $[40, 72)$, soit 192 éléments calculés pour 142 utiles ; le dernier chargement lit un octet au-delà de l'entrée de P_1 (figure 1.2).

1.1.5 Panorama des analyses menées

Pour que le lecteur situe la suite, voici les analyses et ce qu'elles établissent (détails aux chapitres 2 et 3) :

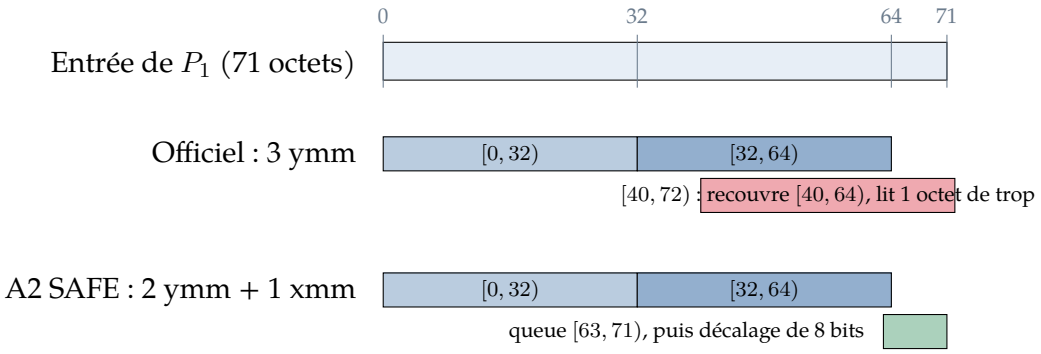


FIG. 1.2 : Chargements pour une entrée de P_1 de MAYO-5 (octets). Le noyau officiel recouvre l'intervalle $[40, 64)$ et dépasse l'entrée d'un octet ; A2 SAFE lit exactement $[0, 71)$.

- **Comptage** (théorique, dans un modèle défini) : borne $\lceil m/32 \rceil$ de `vpshufb` par paire de scalaires et par m-vecteur, réalisation par A2 pour $m \in \{80, 142\}$;
- **Correction et sûreté** : tests de réponses connues (KAT), tests unitaires, bornes de lecture et d'écriture, invariants des tables, indépendance au bourrage ;
- **Mesures de temps** sur une machine physique (compteur TSC, unité statistique : le processus) : gains sur les noyaux isolés, puis sur `mayo_sign`, avec contrôle de l'effet d'ordre et des sondes de mesure ;
- **Analyse statique du code généré** (E1, E1b) : comptage d'instructions et d'écritures dans les boucles produites par GCC 16.2.0, désactivation de passes une à une ;
- **Chaque résultat porte une étiquette** : **[F]** fait établi, **[I]** inférence, **[H]** hypothèse non testée.

1.2 Revue de littérature

1.2.1 Implémentations existantes

Référence C. Arithmétique scalaire, sans `vpshufb`.

AVX2 officiel (MAYO-C). Fichier `src/AVX2/shuffle_arithmetic.h` : macros `MULTIPLY_ACCUMULATE_M_VEC` et `DISENTANGLE_AND_ACCUMULATE_M_VECS`, recouvrement décrit plus haut. Base de ce travail : commit amont `74de637` [15].

***Nibbling MAYO* [7].** Implémentation AVX2 et Cortex-M4 optimisée avec représentation en quartets ; §5.1 : un `vpshufb` fournit 64 multiplications, accumulation séparée impair/pair, et, pour $m = 96$ (MAYO-3 de l'époque), un recouvrement qui « duplique 32 opérations » ; débit d'un `vpshufb` par cycle sur Skylake, deux sur Ice Lake. L'article, lu intégralement, précise le cadre : paramètres du tour 1 ($m = 64$ pour MAYO-1 et MAYO-2, 96 pour MAYO-3, 128 pour MAYO-5), si bien que MAYO-1 et MAYO-5 n'y ont *pas* de queue de vecteur ; mesures sur Skylake (Xeon E3-1245 v5) et Ice Lake (Xeon Gold 6338), compilées avec `clang-14`, Turbo Boost

désactivé, médiane de 10 000 itérations ; tables de multiplication construites par la méthode de [14], appliquée deux fois puis entrelacée ; l'approche par `vpshufb` se distingue de [14] en cherchant *deux* produits par recherche ; seul le recouvrement pour m non multiple de 64 est évoqué (cas MAYO-3), sans traitement dédié de la queue. Le code artefact (`shuffle_arithmetic_\{64,96,128\}.h`) charge à des positions de limbes 0 et 2 (octets $[0, 32)$ et $[16, 48)$) et utilise quatre `vpshufb` par paire pour $m = 96$.

Backend GFNI. Utilise `vgf2p8affineqb` et lit en pleine largeur dans une marge (`MAYO_MVEC_SLACK`) ; hors du cadre de ce mémoire.

1.2.2 Canaux auxiliaires et temps constant

Les attaques temporelles [10] et de cache [11] imposent que les adresses et les branches ne dépendent pas de données secrètes. Dans les noyaux, les tables de multiplication dérivées du vinaigre ne sont que des opérandes de données de `vpshufb` : ni adresses, ni conditions. A2 doit conserver cette propriété ; l'état de la preuve est discuté au section 2.2.

1.3 Évaluation et positionnement

Le tableau 1.2 sépare ce qui était connu, ce qui est postérieur à l'article et ce qu'A2 fait différemment, dans les sources consultées (article §5.1 lu en entier et les parties AVX2 du code ; historique complet de `shuffle_arithmetic.h` dans le dépôt officiel, commits `3c9be9e`, `d81b689`, `11ff0f0`, `4b7cd94`, `fef05b5` et tête de `main` `74de637`).

Aucun `vpshufb` xmm sur une queue n'a été trouvé dans l'artefact de l'article, ni dans l'historique officiel jusqu'à la tête de `main` : les `__m128i` qui apparaissent à partir de `fef05b5` (lignes 548–606) concernent la transposition par décalage de bits de m-vecs et le chargement d'une table de 16 octets diffusée sur 256 bits. **Limite :** l'article n'a été lu intégralement que dans sa section 5.1 et ses parties AVX2 pertinentes ; l'absence de la technique dans ces sources ne prouve pas qu'elle n'existe pas ailleurs.

Tab. 1.2 : Situation d'A2 par rapport aux sources consultées.

Catégorie	Élément	Source
Connu avant A2	`vpshufb` à $t = 1$, deux produits par octet ; séparation impair/pair ; désintrication tardive	[7]
Connu avant A2	Recouvrement de registres pour m non multiple de 64 (MAYO-3, $m = 96$)	[7]
Postérieur à l'article	Généralisation par chargements reculés pour tout $m \leq 192$	code officiel (Round 2)
Différent dans A2	Queue ≤ 16 éléments regroupée dans *un* xmm ([bas|haut] des quartets), *un* `vpshufb` xmm sur la voie basse de la table	ce travail
Différent dans A2	Variante SAFE pour $m = 142$: charge $[63, 71)$ puis décale, aucune lecture hors de l'entrée	ce travail
Différent dans A2	Propriétés vérifiées de la queue (section 2.2)	ce travail
Non revendiqué	Recouvrement, séparation impair/pair, désintrication tardive, schéma $t = 1$, nouveauté absolue	–

Chapitre 2

Méthodologie

Ce chapitre décrit la démarche : la conception d'A2 et son modèle de comptage (section 2.1), les propriétés de correction et de sûreté sur lesquelles elle repose (section 2.2), puis le protocole expérimental et d'analyse statique (section 2.3).

2.1 Conception d'A2 : comptage, réalisation, hypothèses et limites

Ce chapitre répond à QR1 et décrit la réalisation (QR2). Les propositions 2.3 à 2.5 sont des énoncés mathématiques *dans un modèle* ; aucune conclusion de performance n'en est déduite (section 2.1.4).

2.1.1 Modèle de comptage

Définition 2.1 (Données)**.** Un m*-vec* est un vecteur $v \in \mathbb{F}_{16}^m$, m pair, stocké sur $b = m/2$ octets : v_{2i} est le quartet bas de l'octet i, v_{2i+1} le quartet haut. Les K scalaires $c_0, \ldots, c_{K-1}$ sont regroupés en $P = \lceil K/2 \rceil$ paires (complétées par 0 si K est impair). Pour chaque $k < K$ on calcule $\text{acc}_k \leftarrow \text{acc}_k \oplus c_k v$.

Pour la paire p, $T_p[x] = (c_{2p}x) \oplus ((c_{2p+1}x) \ll 4)$; `vpshufb` de largeur $W \in \{16, 32\}$ octets effectue W recherches indépendantes, chacune livrant, pour *un* élément de v, les deux produits de la paire.

Définition 2.2 (Classe $\mathcal{C}$)**.** Algorithmes qui (i) obtiennent chaque produit $c_k v_j$ par une recherche dans une table de 16 octets ; (ii) fournissent à chaque recherche un quartet isolé dans un octet (bit 7 nul) ; (iii) ne font pas deux recherches pour le même couple (élément, paire) hors du recouvrement entre registres ; (iv) utilisent librement XOR, ET, décalages, permutations, chargements. Sont *hors* de $\mathcal{C}$: GFNI, représentations *bitsliced*, tables de 256 octets.

La quantité comptée est $N(\text{algo}, m)$: nombre de `vpshufb` exécutés *par paire de scalaires et par* m*-vec*.

2.1.2 Énoncés

Proposition 2.3 (Borne)**.** *Tout algorithme de $\mathcal{C}$ vérifie $N \geq \lceil m/32 \rceil$.*

Démonstration. Chaque élément v_j ($j < m$) exige au moins une recherche par paire ; une instruction en fournit au plus $W \leq 32$. Donc $32N \geq m$. □

Proposition 2.4 (Schéma officiel avec recouvrement)**.** *Le schéma qui couvre les b octets par $\lceil b/32 \rceil$ chargements de 32 octets, le dernier reculé, et traite chaque registre par deux* `vpshufb` *(impairs, pairs) a $N_{\text{off}}(m) = 2\lceil m/64 \rceil$.*

Démonstration. $\lceil b/32 \rceil = \lceil m/64 \rceil$ chargements, chacun fournissant 32 indices impairs et 32 pairs, soit deux `vpshufb`. □

Proposition 2.5 (Réalisation de la borne si $0 < r \leq 32$)**.** *Écrivons $m = 64q + r$, $0 \leq r < 64$, et $t = r/2$ le nombre d'octets de la queue. Si $0 < r \leq 32$, l'algorithme suivant est dans $\mathcal{C}$ et a $N = 2q + 1 = \lceil m/32 \rceil$:*

1. *pour $g < q$: charger l'intervalle $[32g, 32g + 32)$ et le traiter par deux* `vpshufb` *ymm ;*
2. *pour la queue : charger les t octets de $[32q, 32q{+}t)$, construire $X = [$quartets bas des t octets $\mid$ quartets hauts des t octets$]$ ($r \leq 32$ indices dans un seul registre) et appliquer* un `vpshufb` *;*
3. *réassembler les sorties de la queue (même formule qu'un registre complet, restreinte aux r positions).*

Démonstration. Le décompte est $2q + 1$, et $\lceil m/32 \rceil = 2q + \lceil r/32 \rceil = 2q + 1$ pour $0 < r \leq 32$. □

Conditions de réalisation (à satisfaire par le code, non par le modèle) : (a) le chargement de la queue ne lit que des octets de l'entrée ; (b) si $r \leq 16$ ($t \leq 8$), X tient dans un registre 128 bits et la table utilisée est la *voie basse* de T_p ; si $16 < r \leq 32$, X occupe un registre 256 bits (voie basse = impairs, haute = pairs), ce qui exige l'égalité des deux voies ; (c) le surcoût de préparation de la queue est payé une fois par entrée, celui du réassemblage une fois par paire.

Corollaire 2.6 (Limite exacte)**.** *Pour $m \leq 192$, $\Delta(m) := N_{\text{off}}(m) - \lceil m/32 \rceil = 1$ si $0 < (m \bmod 64) \leq 32$ et 0 sinon. Dans le second cas, le code officiel atteint déjà la borne : aucun algorithme de $\mathcal{C}$ ne peut faire mieux en nombre de* `vpshufb`*.*

TAB. 2.1 : Application aux paramètres actuels.

	m	q	r	N_{off}	$\lceil m/32 \rceil$	Δ	Statut
MAYO-1	80	1	16	4	3	1	implémentée (A2), queue de 8 octets
MAYO-2	64	1	0	2	2	0	sans objet
MAYO-3	108	1	44	4	4	0	impossible dans $\mathcal{C}$
MAYO-5	142	2	14	6	5	1	implémentée (A2), queue de 7 octets + 1 de bourrage

Ce qui est implémenté et ce qui ne l'est pas. Le prototype traite les queues d'au plus 16 éléments ($t \leq 8, r \leq 16$). La variante pour $16 < r \leq 32$ **n'est ni implémentée ni testée** ; elle n'est pas un résultat acquis. Aucun des quatre paramètres actuels n'a $r \in (16, 32]$.

2.1.3 Conception d'A2

2.1.3.1 Structure du noyau

Pour une entrée de P_1 et une ligne d'accumulateurs de K vecteurs, A2 conserve inchangés les $q = \lfloor m/64 \rfloor$ registres ymm complets sur les $32q$ premiers octets, puis traite la queue ainsi :

1. **Préparation (une fois par entrée)** : chargement de 8 octets (`_mm_loadl_epi64`), décalage de 4 bits (`vpsrlw`), entrelacement `vpunpcklqdq` (quartets bas dans les octets 0–7, hauts dans 8–15), ET avec `0x0f` : quatre instructions (cinq dans la variante SAFE, qui ajoute un décalage d'octets).
2. **Produits** : un `vpshufb` xmm avec la *moitié basse* de la table de multiplication déjà chargée, et un `vpxor` xmm dans un accumulateur xmm.
3. **Réassemblage (fin de ligne)** : séparation impair/pair identique à la partie ymm, et lecture-modification-écriture d'*un* limbe de 64 bits (le limbe d'indice $4q = L - 1$, soit 4 pour MAYO-1 et 8 pour MAYO-5).

Dans l'assembleur de GCC 16.2.0, la boucle interne passe donc, par entrée, de 20 à 15 `vpshufb` pour MAYO-1 (10 ymm + 5 xmm) et de 36 à 30 pour MAYO-5 (24 ymm + 6 xmm).

2.1.3.2 Variante par défaut et variante SAFE (MAYO-5)

Pour MAYO-1, $b = 40 = 32 + 8$: la queue est exactement la fin de l'entrée, $[32, 40)$. Pour MAYO-5, $b = 71$ et $64 + 8 = 72$:

- *défaut* : lit $[64, 72)$, soit **un octet au-delà** de l'entrée (comme le noyau officiel, dont le dernier chargement couvre $[40, 72)$) ;
- *SAFE* : lit $[63, 71)$ puis décale le registre de 8 bits vers les poids faibles ; l'octet 63 est éliminé, les octets 64–70 sont placés aux positions 0–6, la position 7 reçoit 0. **Toute lecture reste dans l'entrée.**

La variante retenue pour une éventuelle intégration est SAFE seule. Les quartets de bourrage (positions 142 et 143 de MAYO-5, car $16L = 144$) n'ont pas d'influence sur les positions utiles (section 2.2.4).

2.1.3.3 Préconditions

(P1) P_1 est dans la disposition empaquetée, sans bourrage entre entrées, à l'adresse $P_1 + e \cdot b$; (P2) les accumulateurs ont L limbes avec bourrage de quartets seulement en fin de dernier limbe ; (P3) les tables de multiplication sont construites par la routine officielle (voies égales, lemme 2.7) ; (P4) $m \in \{80, 142\}$: les noyaux A2 ne sont pas

actifs pour les autres paramètres (condition de compilation `MAYO_TAIL_XMM_ACTIVE`) ; (P5) compilation sans `ENABLE_PARAMS_DYNAMIC`.

2.1.4 La borne n'implique pas l'accélération

Une réduction du nombre de `vpshufb` ne garantit pas une réduction du temps : s'y ajoutent le coût de préparation de la queue, les dépendances, les rechargements depuis la pile, les chargements non alignés et le débit des unités. Si seule la contrainte de débit du `vpshufb` (un par cycle sur Skylake [7]) bornait la boucle, le rapport de temps attendu serait N_{A2}/N_{off} : $15/20 = 0{,}750$ (MAYO-1) et $30/36 = 0{,}833$ (MAYO-5).

Tab. 2.2 : Rapport attendu par le seul débit de `vpshufb` et rapport mesuré (S1, A2/officiel, noyaux isolés).

Noyau	Borne « débit seul »	Mesuré (S1)	Lecture
MAYO-1 `P1_times_Vt`	0,750	0,780	plus lent que la borne
MAYO-1 `P1t_times_Vt`	0,750	0,710	plus rapide que la borne
MAYO-5 `P1_times_Vt`	0,833	0,807	plus rapide que la borne
MAYO-5 `P1t_times_Vt`	0,833	0,880	plus lent que la borne

Deux cas sur quatre sont plus rapides que ne le permet le seul comptage de `vpshufb` : d'autres mécanismes contribuent (par exemple moins de rechargements de pile : 11 $\rightarrow$ 5 pour MAYO-1 `P1_times_Vt`, 30 $\rightarrow$ 24 pour MAYO-5), sans que cela soit établi. Le modèle de débit n'explique donc pas à lui seul les gains. Une hypothèse de performance H-P (« sur une microarchitecture où le débit des shuffles est le goulot, le temps est borné inférieurement par $N \cdot P$ cycles par entrée ») ne peut se tester qu'avec des cycles cœur, pas des ticks TSC ; elle est postérieure à ce travail et reste ouverte.

2.2 Propriétés de correction et de sûreté

Ce chapitre répond à la partie « sûreté » de QR2. Chaque propriété porte son étiquette : [C] (déduite du code et de ses invariants, par relecture et assertions de compilation), [E] (vérifiée expérimentalement) ou [H] (hypothèse sur la machine ou le compilateur). Une propriété [E] seule n'est pas une démonstration. Notation : $n_y = \lfloor m/64 \rfloor$ (1 pour MAYO-1, 2 pour MAYO-5), $T_{\text{OFF}} = 32n_y$ (32 ; 64), $T_{\text{LIMB}} = 4n_y$ (4 ; 8).

2.2.1 Bornes de lecture de P_1

L'entrée e occupe les octets $[eb, (e+1)b)$.

- [C] Les n_y chargements ymm lisent $[eb + 32g, eb + 32g + 32)$, $g < n_y$; comme $32n_y \leq b$ ($32 \leq 40$; $64 \leq 71$) ils restent dans l'entrée.
- [C] MAYO-1 : la queue lit $[eb + 32, eb + 40)$, exactement la fin de l'entrée.

- [C] MAYO-5, SAFE : la queue lit $[eb + 63, eb + 71)$ puis décale de 8 bits ; toute lecture reste dans l'entrée.
- [C] (variante par défaut, non retenue) MAYO-5 lit $[eb+64, eb+72)$: un octet au-delà. Pour $e < N_{\text{ent}}-1$, c'est le premier octet de l'entrée suivante ; pour la dernière, c'est l'octet 720 863 du tampon, alloué (zone de $8{\cdot}$`P1_LIMBS_MAX`$=$ 731 016 octets). Des `_Static_assert` imposent $N_{\text{ent}}b +$ OVERREAD $\leq 8{\cdot}$`P1_LIMBS_MAX` et OVERREAD≤ 1.
- [E] Test « garde-page » (fin de P_1 collée à une page interdite, marge 0) : l'officiel et A2 par défaut déclenchent SIGSEGV en MAYO-5 ; A2 SAFE non ; MAYO-1 : aucun des trois. Ce test contrôle la propriété ci-dessus sans la démontrer.

Le noyau officiel de MAYO-5 lit lui aussi 1 octet au-delà (dernier ymm sur les octets 40–71) et celui de MAYO-3 en lit 2 (second ymm sur $[24, 56)$ pour $b = 54$). A2 SAFE ne dépend d'aucune marge.

2.2.2 Bornes d'écriture

Pour la ligne r, l'accumulateur est à acc $+ rKL$ (limbes) ; le vecteur k occupe $[kL, (k + 1)L)$.

- [C] Les stockages ymm écrivent les limbes $[kL + 4g, kL + 4g + 4)$, $g < n_y$, jamais au-delà du limbe $4n_y - 1 \leq L - 1$.
- [C] La queue lit-modifie-écrit un seul limbe de 64 bits, d'indice $T_{\text{LIMB}} = L - 1$, pour $k < K$ (et $k + 1 < K$ pour le second, garde explicite). Toutes les écritures sont donc dans les VKL limbes de `acc`. Le noyau officiel écrit un ymm à $L - 4$: même étendue.
- [E] Un « canari » placé après `acc` n'est pas modifié ; P_1, passé en `const`, n'est pas modifié (copie de l'entrée inchangée).

2.2.3 Invariant des deux voies des tables

A2 lit la *voie basse* de chaque table et suppose qu'elle égale la voie haute.

Lemme 2.7. [C] *Pour tout i, `V_multabs[i]` a ses deux voies de 128 bits égales.*

Justification. (i) Chacune des quatre lignes de 32 octets de `__gf16_mulbase` répète deux fois la même suite de 16 octets (constantes lues dans `arithmetic_common.h`, lignes 12–17) ; (ii) `tbl32_gf16_multab2(b)` est un OU exclusif de ET entre ces lignes et des masques obtenus par `_mm256_set1_epi16(b & 0xf)` décalé et comparé, opérations agissant par élément de 16 bits donc par voie, avec des masques identiques dans les deux voies ; (iii) `mayo_V_multabs` combine deux telles tables par `^` et `_mm256_slli_epi16(.,4)`, également par voie. Par induction l'égalité est conservée.

[E] 272 tables vérifiées par le mode `MAYO_TAIL_XMM_CHECK` (arrêt en cas d'écart) sur les tables passées aux noyaux. C'est une *précondition* d'A2 : si les tables étaient construites autrement (autre routine, autre backend), l'invariant devrait être revérifié.

2.2.4 Influence des quartets de bourrage

Seul MAYO-5 a des quartets de bourrage (positions 142 et 143 ; MAYO-1 : $80 = 5 \cdot 16$).

- [C] *Indépendance positionnelle.* La sortie à la position j ne dépend que des entrées à la position j de chaque m-vec : une recherche lit un index et une table, jamais un autre index ; les décalages `srli/slli_epi16` mélangent deux octets voisins mais chaque résultat est suivi d'un ET avec `0x0f` (ou provient d'un registre déjà masqué).
- [C] Les deux seules différences entre officiel et A2 aux positions de bourrage sont : officiel : produits de l'octet 71 (premier octet de l'entrée suivante) ; A2 SAFE : 0.
- [C] Les noyaux alimentent `Pv`, puis `Vt_times_Pv` produit `VP1V`, consommé par `compute_rhs`, premier consommateur qui mélange des positions ; il annule les quartets de bourrage avant usage (`src/mayo.c`, lignes 50–60, masque $(1 \ll ((m \bmod 16) \cdot 4)) - 1$). Entre les noyaux et `compute_rhs`, les opérations sont positionnelles.
- [E] Tests unitaires (20 000 cas par taille, 8 familles d'entrées) : 0 écart sur les $M/2$ octets utiles et sur les limbes complets après masquage ; sur 30 010 signatures appariées de MAYO-5 (15 005 pour MAYO-1 et MAYO-2), 0 signature différente.

Limite. Cette propriété suppose que `Pv` n'est consommé que par les chemins énumérés, par recherche dans le source du backend AVX2 ; une modification future qui lirait le bourrage de `Pv` devrait être réexaminée.

2.2.5 Temps constant : ce qui est établi et ce qui ne l'est pas

- [C] Bornes de boucle (V, K) et adresses de chargement (eb) ne dépendent que de paramètres publics ; les tables dérivées du vinaigre sont des opérandes de données de `vpshufb`, jamais des adresses ni des conditions. L'officiel a la même structure.
- [C]→à vérifier sur le binaire : A2 introduit, au réassemblage de la queue, deux `vmovq` d'un registre vectoriel vers un registre général (`ar[...] ^= (uint64_t)_mm_cvtsi128_si64(...)`) : des données dérivées du secret transitent par un registre général, uniquement comme opérande d'un XOR vers la mémoire. L'inspection de l'assembleur (script `inspect_asm.py`) ne trouve aucun usage non conforme (adresse, branche, drapeau) : résultat pour *une* version de GCC, pas une garantie générale.
- [E] (indicateur) Valgrind en mode temps constant (données secrètes marquées indéfinies), dans la VM seulement ; il détecte branches et adresses dépendant de données marquées sur les exécutions testées.
- [H] Les instructions `vpshufb`, `vpxor`, `vpand`, décalages, chargements et stockages ont un temps indépendant des données sur les microarchitectures visées (documentation constructeur, non vérifiée ici).

Aucune preuve formelle de temps constant n'est fournie. La réussite des KAT et l'absence de différence fonctionnelle ne démontrent pas le temps constant.

2.2.6 Correction fonctionnelle

KAT officiels réussis pour MAYO-1, 2, 3 et 5 (toutes les sessions), tests de schéma réussis, tests unitaires précédents, zéro signature différente dans toutes les paires A/B, nombre de tentatives de signature identique. La correction est donc établie *sur les entrées testées* ; l'argument général repose sur les propriétés [C] ci-dessus.

2.3 Protocole expérimental

L'objectif est qu'un lecteur disposant du dépôt, des archives et d'une machine comparable puisse refaire chaque analyse et, à des variations de bruit près, chaque mesure. Les commandes figurent à l'annexe A.

2.3.1 Plateforme

Machine physique (toutes les mesures citées au chapitre 3). Portable Intel Core i9-9880H (8 cœurs, 16 threads, microcode `0x104`) ; `systemd-detect-virt` = `none`, drapeau hyperviseur = 0 ; Kali 7.1.5 (démarrage sur disque, double amorçage) ; GCC 16.2.0 (Debian 16.2.0-3), cmake 4.3.4, Python 3.14.8, numpy 2.4.6. Gouverneur `performance`, turbo actif, SMT actif, secteur branché. Processus épinglé sur le cœur 1 (`taskset`). La fréquence instantanée du cœur n'est enregistrée qu'au lancement (de 800 MHz au repos à 4 540 MHz selon les sessions) ; elle n'est pas une fréquence soutenue. Horloge du portable à UTC+2 ; les journaux `machine.txt` sont en UTC.

Environnement virtuel (historique, non utilisé pour les résultats de performance). VM KVM à 2 cœurs, GCC 13.3.0, cmake 3.28.3. Les mesures de cette VM (qui ont orienté les décisions de conception) ne sont jamais présentées comme performances matérielles.

Compilation. Options de la bibliothèque : `-O3 -funroll-loops -march=haswell -maes`, identiques pour les deux versions comparées. Pour l'analyse statique, la commande exacte de chaque objet est lue dans `compile_commands.json`.

2.3.2 Code comparé

Base amont : dépôt officiel `PQCMayo/MAYO-C`, commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`. Les modifications sont appliquées par `git am` dans l'ordre : `0001` (prototype A2), `0002` (sondes de décomposition, inertes par défaut), `0003` (contrôle de l'effet des sondes), `0004` (analyse statique E1, aucun noyau modifié). Les identifiants de commit changent avec `git am` ; les arbres, non (annexe A). L'arbre des mesures S1/S2 est

celui après `0001` ; celui de PH1/CT1 après `0003`. Les noyaux optimisés n'ont pas été modifiés pendant la phase d'analyse.

2.3.3 Protocole de mesure apparié

Pour chaque niveau mesuré (`P1_times_Vt`, `P1t_times_Vt`, `m_calculate`, `mayo_sign`), un processus exécute des paires (officiel, A2) en ordre *aléatoire*, avec la même graine et le même message dans les deux bras. Le temps est lu par `rdtscp` (ticks TSC) ; l'ordre d'exécution de chaque paire est enregistré (`xmm_first`). Un processus comprend 3 001 paires. Les signatures des deux bras sont comparées octet à octet et le nombre de tentatives de signature est vérifié.

2.3.4 Unité statistique et intervalles

Les 3 001 paires d'un même processus ne sont pas des répétitions indépendantes (même état de la machine, même fréquence) : l'IC95 « naïf » calculé sur les paires est de l'ordre de $\pm 0{,}0001$ pour `mayo_sign`, alors que la dispersion entre processus est 5 à 15 fois plus grande. **L'unité statistique est le processus.** Pour chaque niveau et chaque processus on calcule la médiane du rapport A2/officiel ; la valeur rapportée est la médiane de ces médianes, avec un IC95 *bootstrap sur les processus*. Une seule différence entre sessions est testée (S2−S1) : avec deux démarrages, la composante « entre sessions » n'est pas estimable séparément ; on donne l'écart et son IC bootstrap.

Effet d'ordre. Pour chaque processus : médiane du rapport sur les paires « A2 en premier » et sur les paires « officiel en premier » ; l'effet d'ordre est la différence, avec IC bootstrap sur les processus ; il est jugé relativement au gain.

2.3.5 Sessions

Tab. 2.3 : Sessions de mesure sur machine physique (dates UTC).

Session	Date, démarrage	Plan	Objet
S1	19 oct. 21 :58–22 :43 ; un démarrage	4 campagnes × 5 processus × 3 001 paires (graines 1–20)	gains
S2	20 oct. 12 :10 ; nouveau démarrage	5 processus (graines 21–25)	reproductibilité
PH1	21 oct. 19 :13 ; nouveau démarrage	MAYO-5 : 10 processus ; MAYO-1, MAYO-2 : 5	décomposition instrumentée
CT1	21 oct. 19 :28 ; même démarrage que PH1	5 configurations × 10 processus (graines 41–50)	effet des sondes

Les scripts de lancement refusent d'écraser un dossier ou une archive existants ; aucune campagne n'écrase la précédente. S1 et S2 comparent deux démarrages distincts avec le même code, la même machine et le même compilateur. Temps

absolus : S2 est environ 7 % plus lent que S1 pour tous les niveaux, témoins compris ; la cause n'est pas mesurée et les rapports ne sont pas affectés.

2.3.6 Instrumentation de décomposition

Des sondes (`rdtscp` précédé d'un `lfence`) mesurent, pour chaque signature et chaque bras, le temps total, le nombre de tentatives et, pour chaque phase, le temps et le nombre d'appels. Elles sont inertes par défaut : sans la macro correspondante, le code objet de `mayo.c` et `arithmetic.c` est identique à l'original (vérifié par `cmp` et `objdump`). Le patch `0003` ajoute un masque de sondes pour activer individuellement chaque sonde ; les cinq configurations de CT1 sont : `plain` (bibliothèque d'origine, aucune sonde), `none` (sondes compilées, toutes désactivées), `esk` (sonde sur `expand_sk_internal` seule), `mcalc` (sonde sur `m_calculate_M_and_VPV` seule), `all` (toutes les sondes).

2.3.7 Analyse statique du code généré (E1)

Pour chaque construction, l'objet est recompilé par la commande enregistrée, désassemblé (`objdump -d -M intel`) et les boucles de travail sont repérées par leur nombre de `vpshufb` (36 ymm pour l'officiel de MAYO-5 ; 24 ymm + 6 xmm pour A2). Les comptes sont normalisés par le facteur de déroulage de boucle (ici $u = 1$). Le noyau est identifié par le *pas d'adresse sur* P_1 : `add reg,0x47` (entrées de 71 octets, pas d'une entrée) pour `P1_times_Vt`, `imul ...,0x47` pour `P1t_times_Vt` ; les marqueurs de débogage de GCC ne sont pas fiables (les corps identiques sont fusionnés et étiquetés `P1_times_O`). Toute attribution d'une différence de code à une cause exige d'avoir écarté les autres modifications du code (section 2.3.8).

2.3.8 Protocole E1b (analyse statique de l'origine des écritures)

E1b recompile `src/mayo.c` avec *la commande exacte* de E1 plus une seule option supplémentaire, sans aucune mesure de temps. Constructions : `P_none` (sans sondes), `P_all` (toutes sondes), `TAB` (construction des mesures S1/S2). Variantes : `base` (contrôle : doit redonner exactement le code de E1) ; `dump` (sorties `-fdump-tree-optimized`, `-fdump-tree-sra-details`, `-fdump-tree-esra-details`, `-fdump-rtl-expand`, `-fdump-rtl-ira`, `-fopt-info-inline-optimized` ; contrôle : code identique) ; `nosra` (`-fno-tree-sra`) ; `noipasra` (`-fno-ipa-sra`) ; `nodse` ; `nopre` ; `nofre`.

Pour chaque variante, le rapport donne : les boucles de travail avec nombre d'instructions et d'écritures de 32 octets ; le nombre d'instructions de `sign_signature_internal` *hors boucles de travail* ; la distance entre histogrammes de mnémoniques hors boucles par rapport à `base` ; le nombre de lignes GIMPLE mentionnant `temp`. **Règle d'interprétation :** la disparition des écritures dans une variante n'est jamais lue seule ; elle n'est attribuée à une passe que si la distance hors boucles est faible, c'est-à-dire si rien d'autre n'a changé.

TAB. 2.4 : Issues possibles d'E1b et conclusions autorisées (fixées avant exécution).

	Observation (GCC 16.2.0)	Conclusion autorisée / non autorisée
1	Doublon absent avec `-fno-tree-sra` *et* distance hors boucles faible	la passe conditionne le doublon dans cette compilation / non : que `temp[]` en soit la cause sans les dumps
2	Doublon absent mais distance élevée	l'attribution est *non établie*
3	Doublon présent dans toutes les variantes	les passes testées ne le suppriment pas, hypothèse non confirmée / non : que le doublon soit sans cause compilateur
4	`base` $\neq$ code de E1	E1 non reproduit : arrêter l'interprétation

2.3.9 Traçabilité et reproduction

Les données brutes (S1, S2, PH1, CT1) sont déposées avec leurs empreintes SHA-256 (annexe A). Le script de reproduction réextrait les archives, vérifie les empreintes, relance les neuf analyses et compare les sorties : **9 analyses sur 9 identiques** lors de la dernière exécution. La reproduction des *analyses* est exacte ; celle des *mesures* donnera des valeurs voisines mais non identiques (ticks TSC, fréquence, bruit).

2.3.10 Menaces à la validité identifiées a priori

- **Une machine, un compilateur sur machine physique**, trois démarrages ; S2 sur 5 processus.
- **Ticks TSC, non cycles cœur** : la fréquence d'exécution réelle est inconnue ; aucune conclusion sur le débit des unités d'exécution n'est possible.
- **Effet d'ordre** documenté (section 3.3).
- **Les sondes modifient le code** : mesures instrumentées $\neq$ mesures sans sonde.
- **Pas de compteurs matériels** disponibles dans l'environnement d'analyse.
- **Première exécution de la campagne s3 de S1** : CSV bruts non archivés (seule la seconde exécution l'est).
- **Comparaison VM/physique** : le gain de MAYO-5 sur `mayo_sign` était de 7,1 % dans la VM (GCC 13.3.0) ; l'écart avec la machine physique mêle machine et compilateur et n'est pas expliqué.

Chapitre 3

Résultats, interprétation et discussion

Sauf mention contraire, les valeurs sont des rapports A2 SAFE / officiel (médiane des processus), en ticks TSC, sur la machine physique du chapitre 2. Les statuts [F], [I], [H] suivent la convention du l'introduction générale.

3.1 Correction

[F] Les KAT officiels de MAYO-1, 2, 3 et 5 réussissent dans toutes les sessions ; 0 signature différente sur les 30 010 paires de MAYO-5 et les 15 005 de MAYO-1 et de MAYO-2 ; nombre de tentatives identique dans les deux bras. Les propriétés de sûreté sont au section 2.2.

3.2 Gains sur les noyaux et sur la signature (S1, S2)

Tab. 3.1 : Rapport A2/officiel, médiane des processus [IC95 bootstrap entre processus]. S1 : 20 processus ; S2 : 5 processus. Gain = 1 − rapport.

Paramètre	Niveau	S1	S2	Gain S1 / S2
MAYO-1	`P1_times_Vt`	0,7803 [0,7793 ; 0,7817]	0,7814 [0,7794 ; 0,7832]	22,0 / 21,9 %
	`P1t_times_Vt`	0,7099 [0,7060 ; 0,7124]	0,7080 [0,7024 ; 0,7126]	29,0 / 29,2 %
	`m_calculate`	0,8110 [0,8096 ; 0,8118]	0,8102 [0,8080 ; 0,8123]	18,9 / 19,0 %
	`mayo_sign`	**0,8975** [0,8970 ; 0,8979]	**0,8974** [0,8968 ; 0,8990]	**10,3 / 10,3 %**
MAYO-5	`P1_times_Vt`	0,8067 [0,8060 ; 0,8071]	0,8049 [0,8043 ; 0,8076]	19,3 / 19,5 %
	`P1t_times_Vt`	0,8798 [0,8783 ; 0,8818]	0,8792 [0,8770 ; 0,8858]	12,0 / 12,1 %
	`m_calculate`	0,8851 [0,8845 ; 0,8863]	0,8869 [0,8847 ; 0,8874]	11,5 / 11,3 %
	`mayo_sign`	**0,9873** [0,9871 ; 0,9876]	**0,9871** [0,9863 ; 0,9876]	**1,3 / 1,3 %**

[F] Témoins nuls (aucun noyau A2) : MAYO-2 `mayo_sign` 1,0000 / 1,0000, `m_calculate` 1,0001 / 0,9996 ; MAYO-3 `mayo_sign` 1,0000 / 1,0001. Les différences S2−S1 sur les huit lignes de gains sont au plus de 0,0019 en valeur absolue, IC95 contenant 0 (par exemple

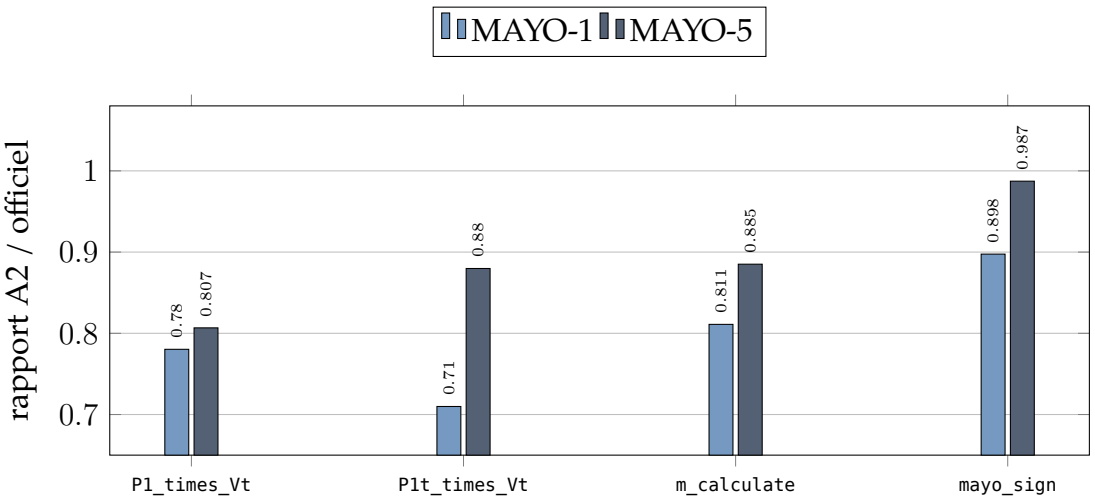


Fig. 3.1 : Rapports A2/officiel mesurés (S1). Le gain se dilue de la couche noyau à la signature complète, fortement pour MAYO-5.

`mayo_sign` : $-0{,}0001$ $[-0{,}0009\,;\,+0{,}0016]$ pour MAYO-1, $-0{,}0003$ $[-0{,}0011\,;\,+0{,}0003]$ pour MAYO-5).

Variabilité à quatre niveaux. Écart-type robuste entre paires (dans un processus) : 0,0015 (S1) et 0,0013 (S2) pour MAYO-1, 0,0114 et 0,0039 pour MAYO-5. Entre processus (dans une session) : 0,0009 (20 proc.) et 0,0010 (5 proc.) pour MAYO-1, 0,0008 et 0,0005 pour MAYO-5. Entre campagnes (S1) : 0,0001 et 0,0002. La dispersion entre processus, et non entre paires, fixe donc l'incertitude.

Gain de MAYO-5 sur la signature. [F] Il est de 1,3 % dans S1, S2, PH1/CT1 et dans la référence, proche du critère de $\approx 1\,\%$ fixé a priori. C'est le seul chiffre de gain de signature de MAYO-5 à retenir (section 3.4).

3.3 Effet d'ordre

[F] Les ordres sont équilibrés : A2 en premier dans 49,97 % des paires de `P1t_times_Vt` de MAYO-1 en S1 $(29\,993/60\,020)$, 50,5 % en S2 ; $P(\text{A2 premier} \mid \text{précédent A2}) = 0{,}5003$.

[F] Il existe un effet générique du protocole d'environ 0,1 % (visible sur MAYO-2 `m_calculate`, où les deux bras sont le même code). L'effet de MAYO-1 `P1t_times_Vt` est environ quinze fois plus grand et propre à ce noyau. Il n'affecte pas `mayo_sign` de MAYO-1 ($\leq 0{,}0001$) ; pour MAYO-5 `mayo_sign` il vaut au plus $0{,}0006$, soit moins de 5 % du gain de $0{,}0127$. Le bras exécuté en second est plus rapide (MAYO-1 `P1t_times_Vt`, S1 : officiel $-0{,}9\,\%$, A2 $-1{,}6\,\%$), et l'effet est plus fort sur A2 d'où l'asymétrie du rapport. [H] La cause n'est pas démontrée et aucune expérience dédiée n'a été faite. Pour ce noyau on cite la moyenne des deux ordres (0,706) en plus de la médiane poolée (0,710).

Tab. 3.2 : Rapport A2/officiel selon l'ordre d'exécution, et écart par processus [IC95].

Niveau	Session	A2 en 1er	officiel en 1er	écart [IC95]
MAYO-1 `P1t_times_Vt`	S1	0,7142	0,6985	+0,0151 [+0,0101 ; +0,0206]
	S2	0,7095	0,7009	+0,0043 [−0,0070 ; +0,0161]
MAYO-1 `mayo_sign`	S1	0,8975	0,8975	−0,0001 [−0,0002 ; 0,0000]
MAYO-5 `mayo_sign`	S1	0,9870	0,9876	−0,0006 [−0,0008 ; −0,0005]
	S2	0,9871	0,9871	0,0000 [−0,0002 ; 0,0000]
MAYO-5 `P1_times_Vt`	S1 / S2	0,8071 / 0,8053	0,8059 / 0,8043	+0,0010 / +0,0010
MAYO-2 `m_calculate` (témoin)	S1 / S2	1,0005 / 1,0004	0,9997 / 0,9994	+0,0010 / +0,0006

3.4 L'anomalie de MAYO-5 : décomposition instrumentée et contrôle

3.4.1 Toutes sondes actives (PH1)

[F] MAYO-5 : une tentative de signature par signature dans les deux bras ; chaque phase est appelée une fois par signature, dont `P1_times_Vt` et `P1t_times_Vt` une fois par tentative. Temps total 1 374 501 (officiel) et 1 274 140 (A2) : rapport 0,9270 [0,9253 ; 0,9283], soit 7,3 %. La différence moyenne de −100 967 ticks se répartit en −100 529 pour `m_calculate` (99,6 %), dont `P1_times_Vt` −54 584 et `P1t_times_Vt` −46 166 ; les autres phases (dont `expand_sk_internal`, `sample_solution`, `compute_A`) sont identiques à ±0,1 %. MAYO-1 : rapport 0,8983 ; MAYO-2 (témoin nul) : 0,9999.

3.4.2 Le gain avec toutes les sondes n'est pas le gain sans sonde (CT1)

Tab. 3.3 : Cinq configurations du même programme (10 processus chacune, graines 41–50, un démarrage). Ticks par signature.

Configuration	Officiel	A2	Rapport [min–max]	Gain
`plain` (aucune sonde)	1 504 741	1 484 504	0,9871 [0,9862–0,9879]	1,29 %
`none` (sondes compilées, désactivées)	1 504 802	1 484 611	0,9868 [0,9860–0,9880]	1,32 %
`esk` (`expand_sk_internal`)	1 504 167	1 484 146	0,9871 [0,9861–0,9879]	1,29 %
`mcalc` (`m_calculate_M_and_VPV`)	1 502 948	1 483 529	0,9877 [0,9865–0,9885]	1,23 %
`all` (toutes les sondes)	1 372 740	1 273 970	0,9276 [0,9261–0,9291]	7,24 %

3.4.3 Mesure directe, sans soustraction de médianes (config. `mcalc`)

[F] Le reste de la signature est identique dans les deux versions. La différence est entièrement dans `m_calculate` *en contexte de signature sans sondes internes* : il coûte 931 000 (officiel) et 912 000 (A2) ticks, contre environ 801 000 et 707 000 isolé ou sondé. Le surcoût de contexte est d'environ 130 000 ticks (officiel) et 205 000 (A2), et la

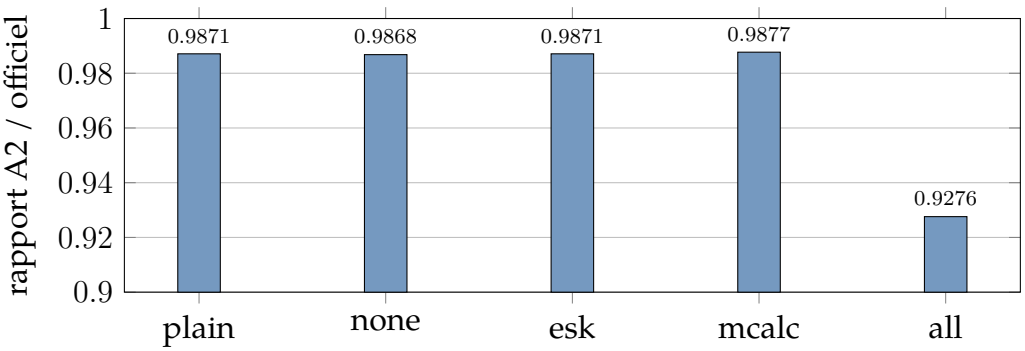


Fig. 3.2 : MAYO-5, rapport `mayo_sign` A2/officiel selon la configuration de sondes (CT1). Seule la configuration « toutes les sondes » change le résultat.

Tab. 3.4 : Décomposition directe de la signature MAYO-5 (ticks par signature).

	Officiel	A2	Différence
`m_calculate` dans la signature	931 453	912 476	$-18\,406$ $[-19\,396 ; -17\,558]$
hors `m_calculate` (total − sonde)	566 917	567 323	$+406$ (0,07 %)
`expand_sk_internal` (config. `esk`)	322 655	322 661	$+6$
`m_calculate` isolé (banc, tampons réutilisés)	801 324	707 314	$\approx -94\,000$ (11,7 %)
`m_calculate`, toutes sondes, dans la signature	801 073	699 775	$\approx -101\,000$

différence de 75 000 ticks entre les deux absorbe l'essentiel du gain isolé. Il disparaît avec toutes les sondes (donc des `lfence` entre les sous-étapes de `m_calculate`) mais pas avec une sonde unique autour de `m_calculate`.

Correction d'une interprétation antérieure. Une première lecture (« le reste de la signature ralentit avec A2 ») résultait d'une soustraction de médianes issues de niveaux de mesure différents ; elle est *retirée*.

3.5 Analyse statique du code généré (E1, GCC 16.2.0)

3.5.1 Comptage de base

[F] Par entrée de P_1, normalisé ($u = 1$) : `vpshufb` officiel 20 (MAYO-1) et 36 (MAYO-5), A2 15 (10 ymm + 5 xmm) et 30 (24 ymm + 6 xmm), conformément au modèle du section 2.1. Dans le banc, instructions de la boucle interne : MAYO-5 `P1_times_Vt` 124 → 106, `P1t_times_Vt` 126 → 110 ; MAYO-1 `P1_times_Vt` 68 → 51, `P1t_times_Vt` 71 → 56. Rechargements de pile : 30 → 24 (MAYO-5 `P1_times_Vt`) et 11 → 5 (MAYO-1). Cette réduction est une explication appuyée sur l'assembleur, pas une preuve que ces instructions expliquent à elles seules le gain.

Tab. 3.5 : Instructions de la boucle interne par entrée de P_1 (MAYO-5) selon la construction.

Construction	`P1` off.	`P1` A2	`P1t` off.	`P1t` A2
banc isolé	124	106	126	110
contexte, toutes sondes	123	105	126	109
contexte, sans toutes les sondes (`ctx_ab`, `esk`, `mcalc`, `none`)	123	135	162	139

3.5.2 Écritures supplémentaires en contexte de signature

[F] La différence entre « sans toutes les sondes » et « toutes les sondes » est *exactement* une écriture de 32 octets (`vmovdqa [rbp-...], ymm`) de plus par accumulateur et par entrée : +36 pour `P1t_times_Vt` officiel (36 accumulateurs), +30 pour chaque noyau A2 (30 accumulateurs). Le reste de la boucle est identique instruction pour instruction. La boucle officielle `P1_times_Vt` n'a pas de doublon. Les écritures vont vers une seconde série d'emplacements consécutifs (pas de `0x20`), distants de `0xbfc0` octets de la première, dont l'adresse est prise ailleurs dans la fonction (`lea`). Deux boucles de 160 instructions (`Ut_times_O`, `m_lambda_times_O`) portent aussi un doublon (123 → 160) dans *les deux* bras. Le cadre de pile dépasse 220 Ko, ce qui interdit de distinguer par le seul décalage un débordement de registres d'un tableau local. Avec GCC 13.3 (VM), aucun doublon n'apparaît. Pour MAYO-1 (GCC 16.2.0), le doublon touche seulement `P1t_times_Vt` (officiel 71 → 91 ; A2 56 → 71).

[F] L'identification de la boucle officielle `P1_times_Vt` dans le contexte repose sur le pas d'adresse `add reg,0x47` et a été contrôlée dans six constructions ; elle donne 123 instructions et 29 écritures dans les contextes officiel, sans sondes et avec toutes les sondes (adresses distinctes), sans doublon. Le tableau 3.5 résume ces comptes.

3.6 Lien quantitatif entre instructions et temps : modèle à un paramètre

[F] Gain statique (P1+P1t) : banc 13,6 %, contexte toutes sondes 14,1 %, contexte sans sondes **3,9 %** (250 → 216 ; 249 → 214 ; 285 → 274 instructions). Gain mesuré de `m_calculate` : 11,5–11,7 % isolé, 12,4 % toutes sondes, **2,1 %** sans sondes (936 031 → 916 206). Dans les trois constructions, l'avantage statique et l'avantage mesuré évoluent dans le même sens.

Modèle. Le temps d'une boucle est supposé proportionnel à son nombre d'instructions par entrée avec un coefficient c commun aux deux bras ($N_{\text{ent}} = 10\,153$). Étalonné sur « toutes sondes » : $c = 635\,681/(249 \cdot 10\,153) = 0{,}251$ tick/instruction. **Prédiction** sans sondes : gain de P1+P1t de $28\,082$ ticks ; **mesuré** : $19\,825$. Le modèle *surestime* le gain d'environ 40 %.

Coût par écriture supplémentaire. À partir de la différence « sans sondes » − « toutes sondes » : officiel $130\,895/365\,508 = 0{,}358$; A2 $210\,808/609\,180 = 0{,}346$ tick par écriture.

Portée de ce rapprochement. La proximité de 0,358 et de 0,346 est un simple test de cohérence (un paramètre, deux observations) ; *elle ne constitue pas une validation du modèle*. Le modèle surestimant le gain, il n'explique pas quantitativement l'écart. La corrélation entre écritures supplémentaires et perte de gain n'est pas une preuve de causalité.

3.7 Analyse statique E1b : résultats (GCC 16.2.0)

E1b a été exécutée sur la machine de mesure (GCC 16.2.0, binutils 2.47) ; l'archive des sorties (SHA-256 `49452120b1b787ad2aed0f30cbcece4f7584a6063768757d92214e264dcd3faa`) est déposée avec le script `e1b_dumps_analyse.py` qui produit les comptes cités ici. Aucune mesure de temps n'est impliquée.

3.7.1 Contrôles

[F] Pour les trois constructions (`P_none`, `P_all`, `TAB`), le code de `sign_signature_internal` recompilé en variante `base` est identique à celui d'E1. La variante `dump` (options de sortie seulement) redonne un code identique (distance hors boucles : 0). Les constructions `TAB` et `P_none` donnent des comptes identiques dans toutes les variantes.

3.7.2 Désactivation de passes, une à la fois

Tab. 3.6 : E1b : boucles de travail selon l'option ajoutée (instructions / écritures de 32 ou 16 octets). « Dist. » : distance entre histogrammes de mnémoniques hors boucles de travail, par rapport à `base`.

Constr.	Variante	Dist.	`P1` A2	`P1t` A2	`P1t` officiel
`P_none`	`base`	0	135 / 52	139 / 52	162 / 65
	`nosra`	1	135 / 52	139 / 52	162 / 65
	`noipasra`	1	135 / 52	139 / 52	162 / 65
	`nodse`	412	135 / 52	139 / 52	162 / 65
	`nopre`	17	135 / 52	139 / 52	162 / 65
	`nofre`	357	135 / 52	139 / 52	185 / 58 (autres boucles modifiées)
`P_all`	`base`	0	105 / 22	109 / 22	126 / 29
	`nosra`	3	105 / 22	109 / 22	126 / 29
	`nodse`	45	105 / 22	109 / 22	126 / 29
	`nopre`	5	105 / 22	109 / 22	126 / 29
	`nofre`	876	**135 / 52**	**139 / 52**	185 / 58
	`noipasra`	1181	aucune boucle de travail reconnue (fonction de 2 892 instr.)		

[F] Aucune des variantes `nosra`, `noipasra`, `nodse`, `nopre` (désactivation de SRA, d'IPA-SRA, de DSE et de PRE) ne supprime le doublon dans `P_none` ni `TAB` : les comptes de boucles sont inchangés (ligne 3 du tableau 2.4). [F] Avec la variante `nofre`, les boucles A2 gardent leur doublon mais le code change profondément ailleurs (distance 357)

et les boucles officielles prennent une autre forme ; dans `P_all`, la même option *fait apparaître* le doublon dans les boucles A2 (135/52, 139/52), mais avec une distance de 876. [I] Ces deux dernières observations n'autorisent *aucune attribution* à la passe FRE, car d'autres parties du code changent en même temps. [F] La variante `noipasra` sur `P_all` modifie la structure de la fonction au point qu'aucune boucle de travail n'est reconnue : aucune lecture n'est possible.

3.7.3 Ce que montrent les sorties de GCC

SRA ne traite pas `temp`, dans aucune des deux constructions. [F] Dans le dump `sra`, `temp` est disqualifié 12 fois dans chacune (8 : « Address taken in a non-call-argument context » ; 4 : « No or inhibitingly overlapping accesses ») ; le dump `esra` ne contient aucune mention. SRA ne peut donc pas expliquer la différence entre `P_none` et `P_all`, ce qui concorde avec le tableau 3.6. L'hypothèse formulée avant E1b (rôle de SRA) est *écartée*.

Les tableaux sont présents dans les deux constructions. [F] Le GIMPLE optimisé déclare les mêmes tableaux locaux dans les deux cas (7 `__m256i temp[36]`, 2 `__m256i temp[24]`, 2 `__m128i tt[6]`).

Ce qui diffère : les stockages dans la boucle. [F] Parmi les blocs de travail du GIMPLE optimisé, 8 contiennent des stockages `temp[i] = ...` (ou `tt[i] = ...`) *dans le corps de la boucle* dans `P_none`, contre 3 dans `P_all`. Dans ces blocs, le nombre de stockages par itération est égal au nombre d'accumulateurs (24 + 6 pour A2, 36 pour l'officiel), il n'y a *aucune lecture* de `temp[]` dans la boucle, et les accumulateurs circulent par des PHI (variables `prephitmp_N`). Dans `P_all`, 9 blocs utilisent des scalaires `temp_I_lsm_N` sans stockage de tableau (144 variables `temp_I_lsm` déclarées dans `P_none`, 300 dans `P_all`).

Lien avec l'assembleur. [F] La boucle A2 de `P1_times_Vt` compte 52 écritures dans `P_none` (42 de 32 octets et 10 de 16 octets) et 22 dans `P_all` (18 et 4). Les 22 écritures de `P_all` visent exactement les mêmes emplacements de pile que 22 des 52 de `P_none` ; les 30 autres sont 24 écritures de 32 octets (pas de `0x20`) et 6 de 16 octets (pas de `0x10`), soit la forme de `temp[24]` et de `tt[6]`. [I] L'identification des 30 écritures supplémentaires aux stockages de `temp[]/tt[]` du GIMPLE repose sur ce décompte et sur la forme des tableaux, pas sur une correspondance de symboles.

3.7.4 Interprétation autorisée

[I] Dans la construction sans toutes les sondes, GCC remplace les lectures de `temp[]` dans la boucle par des valeurs circulant dans des PHI, mais *n'enlève pas les stockages de la boucle* ; le tableau, dont l'adresse est utilisée après la boucle, est donc mis à jour à chaque itération en plus des emplacements de débordement des accumulateurs. Dans la construction avec toutes les sondes, les accumulateurs sont traités comme des scalaires (suffixe `_lsm`) et le tableau n'est écrit qu'à la sortie de la boucle. [I] Cela correspond à l'hypothèse de départ (`temp[]` conservé en mémoire en plus des scalaires), mais *sans SRA*.

[H] Le suffixe `_lsm` est, d'après la convention de nommage de GCC (non revérifiée dans ses sources ici), celui de la *motion de stockages hors des boucles* (passe `lim`) et le préfixe `prephitmp` celui de la passe PRE ; ces rapprochements sont à confirmer par les dumps de ces passes, qu'E1b n'a pas produits. **Non établi :** pourquoi cette motion de stockages a lieu dans `P_all` et pas dans `P_none`, quelle passe en décide, et si le doublon se supprime sans autre changement de code. Le lien entre ces stockages et la perte de gain mesurée reste, comme au §3.6, une corrélation.

3.8 Comparaison avec la VM

[F] Dans la VM (Xeon virtualisé, GCC 13.3.0), le gain de `mayo_sign` pour MAYO-5 était de 7,1 %, proche du 7,3 % mesuré avec toutes les sondes sur la machine physique. [H] L'écart avec 1,3 % sans sonde mêle machine et compilateur ; il n'est pas expliqué. Ce rapprochement est mentionné sans être exploité.

3.9 Discussion critique

Ce chapitre sépare ce qui est établi, ce qui est inféré et ce qui reste hypothèse, puis discute les limites et les pistes écartées.

3.9.1 Bilan épistémique

3.9.2 Pourquoi le gain se dilue

Le gain d'un noyau ne se reporte sur la signature qu'au prorata de son poids. Pour MAYO-5 sans sondes, `m_calculate` pèse environ $931\,453/1\,504\,741 \approx 62\,\%$ de la signature et son gain en contexte est de $18\,406$ ticks, soit $1{,}2\,\%$ de la signature : c'est, à la précision des mesures, le gain de 1,3 % observé. Pour MAYO-1, le rapport entre gain de signature (10,3 %) et gain de `m_calculate` (18,9 %) donne un poids implicite d'environ 54 % [I]. La question n'est donc pas la fraction de la signature occupée par `m_calculate`, mais pourquoi son gain tombe de 11,7 % (isolé) à 2,0 % (en contexte sans sondes) pour MAYO-5, alors qu'il reste de 18,9 % pour MAYO-1 [F].

3.9.3 Ce que la concordance statique/dynamique prouve et ne prouve pas

Le désassemblage donne un fait solide : le nombre d'instructions de la boucle diffère selon la construction, de façon asymétrique. Les temps varient dans le même sens. Trois précautions s'imposent.

1. **Un modèle à un paramètre qui surestime de 40 %** le gain ne peut pas servir de validation ; son accord sur le coût par écriture entre les deux bras (0,358 contre 0,346) est un contrôle de cohérence, non une preuve.

2. **Corrélation, non causalité** : les constructions qui diffèrent par le doublon diffèrent aussi par la présence de `lfence` entre sous-étapes, par la taille de la fonction et par l'allocation de la pile. Seule une expérience qui supprime le doublon *sans autre changement de code* permettrait de conclure (cf. la conclusion générale).

3. **Ticks et non cycles** : le « coût par écriture » de 0,35 tick équivaut, à une fréquence soutenue non mesurée de 3,5–4,5 GHz pour un TSC de 2,4 GHz, à un ordre de grandeur de 0,5–0,65 cycle ; la compatibilité avec un débit d'une écriture par cycle est une remarque [H], pas un résultat.

Plusieurs raisons possibles d'une surestimation du modèle sont envisageables (toutes [H]) : le coût des instructions n'est pas homogène (un `vpshufb` et une écriture n'ont pas le même coût) ; des boucles hors P1/P1t portent aussi un doublon (`Ut_times_0`, `m_lambda_times_0`) dont les nombres de tours ne sont pas connus ici ; les chargements sur la même ligne de cache peuvent se recouvrir.

3.9.4 Pourquoi MAYO-1 et MAYO-5 se comportent différemment

[I] Pour MAYO-1, le doublon ne touche que `P1t_times_Vt` dans les deux bras à un rapport voisin ($71/91 = 0{,}78$ contre $56/71 = 0{,}79$ dans le banc) : l'avantage statique est conservé (0,770 banc, 0,774 contexte), en accord avec un gain de signature qui ne s'effondre pas. Pour MAYO-5, le doublon touche les deux noyaux d'A2 mais un seul de l'officiel. Cette lecture est compatible avec les faits mais elle reste une inférence, pour la raison donnée ci-dessus.

3.9.5 Pistes explorées puis écartées

Les pistes suivantes ont été explorées dans la VM avant la machine physique ; elles sont consignées comme hypothèses éliminées (avec leurs limites).

Fusion de `P1_times_Vt` et `P1t_times_Vt` en un seul parcours. Idée : lire P_1 une fois au lieu de deux. Le nombre de lectures d'entrées est bien divisé par deux (MAYO-1 : 6 480 → 3 240 ; MAYO-5 : 20 306 → 10 153) mais, avec accumulateurs simultanés (option 1), le temps de `mayo_sign` augmente de 4,9 à 10,4 % (MAYO-1) et de 5,3 à 7,0 % (MAYO-5) ; avec fermeture immédiate (option 2), de 67 à 78 % (MAYO-1, MAYO-3, MAYO-5). L'option 1 exige $V_{\max}$ accumulateurs, soit +49 Kio (MAYO-1) à +159 Kio (MAYO-5) de pile. Correction vérifiée (KAT et 5 000 essais indépendants par paramètre sans écart). Interprétation : le travail SIMD est inchangé et la pression mémoire des accumulateurs dispersés l'emporte sur la réduction des lectures. Piste close ; mesures dans la VM uniquement.

Transposée de P_1. Un diagnostic sur la représentation et le motif d'accès à P_1 conclut que la disposition joue un rôle important dans les 29–31 % de gain théorique maximal observés sur MAYO-3 et MAYO-5, sans attribution quantitative au cache

(pas de compteurs matériels), et que le coût de construction l'emporte sur le gain. Piste close.

Traitement par lots dans EF. Piste fermée ; patch, banc et données archivés. Une anomalie de dérive entre processus sur MAYO-2 a été documentée : l'amplitude d'un rapport à un seul niveau dépend du régime du processus, ce qui impose de lire les rapports avec la dérive de la phase correspondante.

3.9.6 Limites de la validation

Une seule machine ; un seul compilateur sur la machine physique ; trois démarrages ; S2 sur cinq processus ; gain de MAYO-5 proche du critère de 1 % ; sondes complètes qui modifient le comportement ; pas de cycles cœur ; pas de compteurs matériels ; Clang non examiné ; aucune preuve formelle de temps constant ; les propriétés de bourrage reposent sur une énumération des consommateurs de `Pv`. Par ailleurs les mesures de l'article de référence ont été faites avec `clang-14` sur Skylake et Ice Lake et avec des paramètres du tour 1 ; elles ne sont pas directement comparables aux nôtres (GCC 16.2.0, i9-9880H, paramètres du tour 2).

3.9.7 Sur l'intégration dans MAYO-C

Aucune décision d'intégration n'est prise ici. Les éléments à peser sont : le gain de 10,3 % pour MAYO-1 (robuste sur deux démarrages) ; le gain de 1,3 % pour MAYO-5 (proche du critère, dépendant du code généré) ; les préconditions (voies égales, $m \in \{80, 142\}$) ; l'état de la preuve de temps constant (indicateurs seulement) ; la dépendance au compilateur. Un jugement d'intégration plus solide exigerait au moins une seconde machine ou un second compilateur.

Tab. 3.7 : Établi, inféré, hypothétique.

Statut	Énoncé	Appui
[F]	Nombre de `vpshufb` par entrée : 20→15 (MAYO-1), 36→30 (MAYO-5)	assembleur, preuve (prop. 2.5)
[F]	Gains de noyaux 12–29 % ; `mayo_sign` 10,3 % (MAYO-1), 1,3 % (MAYO-5) sur cette machine et ce compilateur	S1, S2, PH1, CT1
[F]	Le reste de la signature est identique entre bras ; la différence est dans `m_calculate` en contexte	mesure directe (`mcalc`)
[F]	Avec toutes les sondes, le gain de MAYO-5 est 7,3 % ; sans, 1,3 %	CT1
[F]	GCC 16.2.0 émet une écriture de 32 octets supplémentaire par accumulateur en contexte sans toutes les sondes, asymétrique entre bras	désassemblage E1
[I]	L'asymétrie de ces écritures (+60 pour A2, +36 pour l'officiel sur P1+P1t) explique l'essentiel de la perte de gain de MAYO-5 en contexte	concordance statique/mesure
[H]	Le coût passe par le débit des écritures plutôt que par les dépendances écriture–lecture ou conflits d'adresses	non testé
[F]	Dans le GIMPLE optimisé, sans toutes les sondes, des stockages `temp[i]=/tt[i]=` restent dans 8 blocs de travail (3 avec toutes les sondes) ; SRA ne traite `temp` dans aucune construction	E1b (dumps `sra`, `optimized`)
[I]	Les 30 (A2) ou 36 (officiel) écritures supplémentaires de l'assembleur sont ces stockages	décompte et forme des tableaux
[H]	Une passe de motion de stockages (`lim`) décide de les sortir de la boucle ou non ; elle les sort avec toutes les sondes seulement	dumps non produits
[H]	Les sondes modifient la structure du code et donc la décision de GCC	cohérent avec E1 et E1b, non démontré

Contributions effectives

Ce mémoire a étudié, sur MAYO-C, le traitement des queues de m-vecteurs dans les noyaux AVX2 `P1_times_Vt` et `P1t_times_Vt`. Les contributions établies sont les suivantes.

1. Un **comptage rigoureux** dans un modèle défini : borne $\lceil m/32 \rceil$, schéma officiel $2\lceil m/64 \rceil$, et caractérisation exacte des m pour lesquels un gain de `vpshufb` est possible ($0 < m \bmod 64 \leq 32$). MAYO-1 et MAYO-5 en bénéficient, MAYO-2 et MAYO-3 non.
2. Une **réalisation** (A2) pour les queues d'au plus 16 éléments, avec une variante SAFE qui ne lit rien au-delà de l'entrée de P_1, et un ensemble de propriétés de lecture, d'écriture, de tables et de bourrage, avec leurs preuves de code et leurs tests.
3. Des **gains mesurés** sur machine physique avec une unité statistique explicitée : 12–29 % sur les noyaux, 10,3 % sur `mayo_sign` de MAYO-1, 1,3 % sur celui de MAYO-5, reproduits sur deux démarrages.
4. La **localisation** de la perte de gain de MAYO-5 dans `m_calculate` en contexte de signature, et la mise en évidence d'écritures supplémentaires asymétriques dans le code généré par GCC 16.2.0, concordantes avec la perte ; l'analyse statique E1b les identifie à des stockages de tableaux d'accumulateurs maintenus dans la boucle et écarte la passe SRA comme cause.
5. Un **dépôt reproductible** (scripts, données brutes, empreintes) dont les neuf analyses se reproduisent à l'identique.

Limites

Le gain de signature de MAYO-5 est faible et dépend de la décision du compilateur. Le lien causal entre écritures supplémentaires et perte de gain n'est pas établi et le modèle quantitatif le surestime d'environ 40 %. Les mesures proviennent d'une machine et d'un compilateur ; elles sont en ticks TSC. Le temps constant n'est étayé que par des indicateurs. La variante pour queues de 17 à 32 éléments n'est pas implémentée. Aucune nouveauté absolue n'est revendiquée.

Questions ouvertes et perspectives

Q1 – Décision du compilateur. Comprendre pourquoi GCC 16.2.0 laisse ces stockages dans la boucle sans toutes les sondes et les sort avec toutes les sondes. E1b a écarté SRA, DSE, PRE et IPA-SRA comme causes uniques (désactivés un à un, le doublon persiste) et n'autorise aucune attribution à FRE (le code change ailleurs). La suite naturelle, restant statique, est de produire les dumps des passes de motion de stockages et de PRE (`-fdump-tree-lim2-details`, `-fdump-tree-pre-details`); elle n'a pas été lancée et attend l'accord de l'encadrant.

Q2 – Causalité. Si, et seulement si, un moyen de supprimer le doublon sans autre modification de code est identifié, comparer les temps avec et sans doublon à code constant. Les expériences E2 à E5 initialement envisagées restent suspendues jusqu'à la décision de l'encadrant.

Q3 – Robustesse. Reproduire l'analyse statique avec Clang et d'autres versions de GCC ; mesurer sur une seconde machine, idéalement avec compteurs matériels et en cycles cœur (hypothèse de performance H-P).

Q4 – Extension. Implémenter et valider la variante pour $16 < r \leq 32$; examiner les paramètres hors de la famille actuelle.

Q5 – Assurance. Vérification formelle ou outillée du temps constant du binaire.

Q6 – Intégration. Décision en fonction des réponses ci-dessus et des critères de l'encadrant.

Références

[1] P. W. Shor, *Algorithms for quantum computation : discrete logarithms and factoring*, FOCS 1994.

[2] J. Patarin, *The oil and vinegar signature scheme*, Dagstuhl Workshop on Cryptography, 1997.

[3] A. Kipnis, A. Shamir, *Cryptanalysis of the Oil and Vinegar Signature Scheme*, CRYPTO 1998.

[4] A. Kipnis, J. Patarin, L. Goubin, *Unbalanced Oil and Vinegar Signature Schemes*, EUROCRYPT 1999.

[5] W. Beullens, *Breaking Rainbow Takes a Weekend on a Laptop*, CRYPTO 2022.

[6] W. Beullens, *MAYO : Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*, IACR ePrint 2021/1144 (SAC 2021).

[7] W. Beullens, F. Campos, S. Celi, B. Hess, M. J. Kannwischer, *Nibbling MAYO : Optimized Implementations for AVX2 and Cortex-M4*, IACR ePrint 2023/1683 (version consultée).

[8] NIST, *FIPS 203, 204, 205 : normes de chiffrement et de signature post-quantiques*, août 2024.

[9] NIST, *Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process*, 2022.

[10] P. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, CRYPTO 1996.

[11] D. J. Bernstein, *Cache-timing attacks on AES*, 2005.

[12] V. Arlazarov, E. Dinic, M. Kronrod, I. Faradzev, *On economical construction of the transitive closure of an oriented graph*, Soviet Math. Doklady 11, 1970.

[13] M. R. Albrecht, G. V. Bard, C. Pernet, *Efficient Dense Gaussian Elimination over the Finite Field with Two Elements*, arXiv :1111.6549, 2011.

[14] W. Beullens, M.-S. Chen, S.-H. Hung, M. J. Kannwischer, B.-Y. Peng, C.-J. Shih, B.-Y. Yang, *Oil and Vinegar : modern parameters and implementations*, IACR TCHES 2023(3).

[15] Dépôt officiel `PQCMayo/MAYO-C`, commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`.

Annexe A

Inventaire des archives et reproduction

État au 22 octobre 2026 (dernier inventaire vérifié) ; les ajouts postérieurs (E1b, identification des noyaux) sont dans le dépôt `mayo-a2-thesis`. Toutes les empreintes sont des SHA-256.

A.1 Versions définitives à citer

Fichier	SHA-256	Contenu
`A2_partie1.tar.gz`	`cbc0cc297ee0a0790b3cb7513d0384c6c323cfe26b6180b93fc7bd1b115477a6`	notes, code, scripts, analyses, brut S2
`A2_partie2.tar.gz`	`d8dbf1b169f249c720a6e74eb0b9c6e6c9fe44eae6087ef6fe188aa21ab0fcab`	brut PH1 et CT1
`A2_partie3.tar.gz`	`00d12e885de273e7851b270f8b53b780eb52fd5bbbe969b1dfe73402b687ac33`	brut S1
patch `0001`	`e6c03362cbf974cf750f340453de874d667b5670a7e60eddafa54b35e29e0cea`	prototype A2
patch `0002`	`751ec21fbd059e51a4b54f193f3e346a889d4b361cfdf78a3ab77c3e891b6280`	sondes
patch `0003`	`b590a81d479fd4a32883a60efe4bb5525fb9f45bf92a1acd70f60b265596112d`	contrôle des sondes
patch `0004`	`9c460d0de2193805f3b0c1716f62bf1390ca543778ccfb0022d995ef254a10c6`	analyse statique E1
`MAYO-C-A2_source_ e72d407.tar.gz`	`e2252a0b51ab827bd097e5a0df73dcbb7ca13539af82b1e728737c58800f6833`	arbre S1/S2
`MAYO-C-A2-phases-v2_source.tar.gz`	`ecb0941b086affb7189151af4b58355acb39edd4de8d7089b34ac7328cf19ef3`	arbre PH1/CT1
`E1_kit.tar.gz`	`efe0b71ca5dc1553b0c0a841dc59096216a214080d988ba32731cbe2fcde18f3`	`run_E1.sh`, `e1_analyze.py`

Archive E1b (sorties de compilation, dumps GCC, désassemblages) : `e1b_gcc16.tar.gz`, SHA-256 `49452120b1b787ad2aed0f30cbcece4f7584a6063768757d92214e264dcd3faa`.

Archives brutes internes : S1 `ebc01dfe`…, S2 `b1a69769`…c723, PH1 `e1a88aec`…98cc, CT1 `0506c2b5`…92de. Les archives de la phase VM (`A2_donnees_et_journaux`,

`tail_aware_etude`, `archive_piste_batch_EF`, etc.) sont de l'*historique* et ne doivent jamais être présentées comme performances matérielles.

A.2 Identification exacte du code

Le code est déterminé par l'arbre git (indépendant de l'horodatage des commits). Base : `PQCMayo/MAYO-C` commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`, puis `git am` des patchs dans l'ordre. Arbres attendus : après `0001` `87d1729c`…(S1, S2) ; après `0002` `c8c37c97`… ; après `0003` `92bcede1`…(PH1, CT1) ; après `0004` `bf6955da`….

```
git clone https://github.com/PQCMayo/MAYO-C && cd MAYO-C
git checkout 74de637a7c4a7f9fafa888183bc05f5cf920cd2d
git am 0001-*.patch 0002-*.patch 0003-*.patch 0004-*.patch
git rev-parse HEAD^{tree}   # attendu : bf6955dae657abb087b58e13e314b946e0d283fe
```

A.3 Reproduire les analyses

Prérequis : Linux, `python3` avec `numpy`, `tar`, `sha256sum`. Les trois archives dans un dossier D :

```
sh reproduire_analyses.sh D travail_neuf
```

Le script vérifie les empreintes, relance les neuf analyses et affiche `OK` si la sortie est identique (résultat obtenu : 9 sur 9). Les sorties couvrent la synthèse S1/S2 (`synthese_sessions.py`), la comparaison (`compare_sessions.py`), la décomposition (`phases_analyze.py`), le contraste avec/sans sondes (`phases_vs_ref.py`) et le contrôle CT1 (`controle_analyze.py`).

A.4 Refaire les mesures

Conditions : Linux, AVX2, GCC $\geq$ 13 (mesures : 16.2.0), cmake, `python3+numpy`, gouverneur `performance` (`sudo cpupower frequency-set -g performance`), cœur 1 dédié, aucun autre travail pendant l'exécution.

Étape	Commande (racine de l'arbre correspondant)
S1, S2	`SESSION=<étiquette> sh run_session_independante.sh`
PH1	`SESSION=<étiquette> sh test/run_phases.sh`
CT1	`SESSION=<étiquette> sh test/run_controle.sh`
E1	`SESSION=<étiquette> sh test/run_E1.sh`
E1b	`python3 -I e1b.py <dossier_E1> <dossier_neuf>`

Chaque `machine.txt` enregistre date UTC, noyau, compilateur, microcode, gouverneur, `boot_id` et alimentation. La reproduction des *mesures* donnera des valeurs voisines et non identiques.

Annexe B

Identification des noyaux dans le code généré (GCC 16.2.0, MAYO-5)

Identification par le pas d'adresse sur P_1 (section 2.3). « Écritures » : écritures de 32 octets dans le corps de boucle.

Tab. B.1 : Instructions / écritures de 32 octets des boucles de travail selon la construction.

Construction	`P1` off.	`P1` A2	`P1t` off.	`P1t` A2
`ctx_official`	123 / 29	–	162 / 65	–
`ctx_ab`, `none`, `esk`, `mcalc`	123 / 29	135 / 52	162 / 65	139 / 52
`ctx_phases` (toutes sondes)	123 / 29	105 / 22	126 / 29	109 / 22

Les différences « sans toutes les sondes » moins « toutes sondes » sont : A2 `P1` $+30$ ($52-22$), A2 `P1t` $+30$, officiel `P1t` $+36$ ($65-29$), officiel `P1` 0.